UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 01

## DevOps: Overview + Comunidades

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda do encontro

- 19:30 — Abertura, boas-vindas e "Energia Matera"
- 19:35 — Parte 1: O Mindset DevOps
- 20:20 — Intervalo
- 20:35 — Parte 2: Apoio à configuração do ambiente + roteiro de testes
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Objetivo de hoje: entender por que o DevOps nasceu, a dor que ele cura e sair com o ambiente funcionando.**

📖 Apareceu uma palavra nova? Consulte o glossário do repositório do bootcamp.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

# O Mindset DevOps

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# A história de uma rivalidade

Era uma vez duas equipes na mesma empresa:

- **Dev (Desenvolvimento):** quer entregar funcionalidades novas, rápido.
- Para o Dev, mudança é o seu trabalho.
- **Ops (Operações):** quer o sistema estável, no ar, sem sustos.
- Para o Ops, mudança é o seu maior risco.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# "No meu ambiente funciona"

- O Dev "joga o código por cima do muro" para o Ops.
- O Ops segura o deploy com medo de quebrar tudo.
- Cada incidente vira caça ao culpado.
- Entregas que deveriam levar dias levam meses.

💬 **Interação: quem aqui já ouviu (ou disse) "no meu ambiente funciona, o problema é do servidor"?**

Deploy = colocar uma versão do sistema no ar (voltamos a esse termo daqui a pouco).

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que é DevOps, afinal?

É a decisão de derrubar o muro entre Dev e Ops e fazer as duas pontas trabalharem como **um time só**.

Um time responsável junto pelo produto: do código à produção.

**DevOps não é um cargo, nem uma ferramenta — é uma cultura.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Pilares: cultura e empatia

- **Cultura e comunicação:** todos falam a mesma língua e olham para o mesmo objetivo.
- **Empatia e quebra de silos:** o Dev entende a dor de operar...
- ...e o Ops entende a dor de entregar.

Silo = cada equipe fechada em si mesma, sem conversar com as outras.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Pilares: automação e blameless

- **Automação:** tudo que é repetitivo e manual vira código (spoiler das próximas aulas!).
- **Cultura blameless (sem culpa):** quando algo quebra, ninguém sai procurando culpado.
- **Post-mortem:** a análise feita depois de um incidente — sem culpados.

**A pergunta certa é "o que no processo permitiu esse erro?" — nunca "quem errou?"**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O ciclo de vida DevOps ♾️

O famoso "loop infinito": o trabalho não termina quando o sistema vai ao ar — ele volta para o planejamento.

Agora vamos desmistificar os termos que você vai ouvir o bootcamp inteiro.

**Planejar → Codificar → Buildar → Testar → Liberar → Implantar → Operar → Monitorar → (e volta ao Planejar)**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Jargão traduzido: CI e CD

- **CI — Integração Contínua**
- Integrar e testar o código várias vezes ao dia, automaticamente.
- **CD — Entrega Contínua**
- Deixar cada versão pronta para ir ao ar com um clique (ou nenhum).

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Jargão: pipeline, build, deploy

- **Pipeline / Esteira:** a sequência automatizada de etapas do código até a produção.
- **Build:** montar/empacotar o programa a partir do código.
- **Deploy:** colocar uma versão do sistema no ar.

**Guarde esses cinco termos: eles aparecem em todas as aulas daqui para frente.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Por que o mercado paga bem?

Empresas que dominam DevOps entregam **mais rápido, com menos erro e mais segurança**.

- Na Matera, sistemas financeiros processam bilhões de transações.
- Sem automação, cultura e monitoramento, isso simplesmente não funciona em escala.

**Profissionais que sabem construir e operar essas esteiras são disputados.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Comunidades: ninguém aprende sozinho

- O ecossistema é gigante e muda rápido.
- Comunidade é como você se mantém atualizado.
- Onde participar: comunidades locais e online de DevOps e Cloud.
- Eventos e meetups.
- Os canais deste bootcamp.

Meetup = encontro (presencial ou online) de uma comunidade para trocar experiências.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Nosso combinado

Usem os canais do bootcamp!

- Dúvida de ambiente? Chame um monitor no canal `#setup-ambiente`.
- Quem pergunta aprende e ajuda os outros a aprender.

**Pergunta boba não existe.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Ambiente funcionando é a primeira entrega!

Na prática de hoje, vamos garantir que **todo mundo** saia com a VM Ubuntu validada.

- A VM (máquina virtual) é o nosso laboratório para as próximas 9 aulas.
- Depois do intervalo: roteiro de testes + primeiros comandos no terminal.
- Travou em algum passo? Os monitores estão de plantão.

Já deixe a VM ligando durante o intervalo — voltamos às 20:35.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Validando seu laboratório

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Objetivo da prática

**Objetivo:** garantir que sua VM Ubuntu está pronta para o bootcamp e dar seus primeiros passos no terminal.

**Pré-requisito:** ter seguido os guias de `setup/` (VirtualBox, VM Ubuntu e roteiro de testes).

- Onde: no terminal do **seu computador**, conectado à VM por SSH.
- Travou em algum passo? Este é o momento de pedir ajuda — os monitores estão de plantão!

SSH = conexão remota e segura ao terminal da VM; é assim que vamos usar a VM em todas as aulas.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — Ligue a VM e faça login

- Abra o VirtualBox, selecione a VM `bootcamp-devops` e clique em **Iniciar**.
- Faça login com o usuário `aluno` e sua senha (ela não aparece enquanto você digita).
- Depois, abra o terminal do **seu computador** e conecte-se por SSH:

```
ssh aluno@localhost -p 2222
```

Resultado esperado: o prompt `aluno@bootcamp:~$` na VM e, depois do `ssh`, também na janela do seu terminal.

REALIZAÇÃO:
Instituto MATERA

# Passo 2 — Roteiro de testes (1/3)

- Testes 1 e 2 (login e SSH) você já fez no Passo 1 — daqui em diante, tudo pelo SSH.
- Teste 3: teclado. `~` é tecla morta (aperte `~` e depois Espaço); `|` é Shift + `\`.
- Teste 4: versão do Ubuntu.

```
echo "teclado: ~ | / > ç"
lsb_release -a
```

Resultado esperado: `teclado: ~ | / > ç` igualzinho, com todos os símbolos; `Description: Ubuntu 24.04.x LTS`.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

# Passo 2 — Roteiro de testes (2/3)

- Testes 5 e 6: internet, DNS (nome → endereço) e HTTPS.
- Rode um comando por vez e compare com o resultado esperado.

```
ping -c 4 google.com
curl -I https://www.google.com
```

Resultado esperado: `4 received, 0% packet loss`; primeira linha `HTTP/2 200`.

# Passo 2 — Roteiro de testes (3/3)

- Testes 7 e 8: `apt` e disco.
- `apt` pede sua senha; avisos em amarelo são OK, só mensagens de erro contam como falha.
- Para encerrar a sessão SSH, digite `exit`.

```
sudo apt update
df -h /
```

Resultado esperado: apt sem erros; `Avail` (ou `Disp.`) com pelo menos 10G.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 3 — Deixe sua marca 🖋️

- `echo` é o comando que "fala" no terminal: ele repete a frase de volta.
- `whoami` mostra quem você é para o sistema.
- `uptime` mostra há quanto tempo a máquina está ligada.

```
echo "Meu primeiro comando no bootcamp DevOps!"
whoami
uptime
```

Resultado esperado: o terminal repete a frase; `whoami` responde `aluno`; `uptime` mostra o tempo que a VM está ligada.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 4 — Compartilhe a vitória 🎉

- Digite o comando, tire um print (captura de tela) do resultado e poste no chat/comunidade.
- `hostname` mostra o nome da máquina e `whoami`, o seu usuário.
- O terminal montou a frase para você.

```
echo "Ambiente pronto! $(whoami) reportando da VM $(hostname)"
```

Resultado esperado: `Ambiente pronto! aluno reportando da VM bootcamp`

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢 (1/2)

- VM não liga / erro de virtualização → seção *Problemas comuns* do guia do VirtualBox.
- Login não funciona → confira usuário/senha; esqueceu a senha? veja o guia da VM.
- `Connection refused` no SSH → OpenSSH não instalado (passo 6) ou porta 2222 (passo 7.1).

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢 (2/2)

- `Permission denied` no SSH → senha errada (nada aparece ao digitar) ou usuário errado.
- Algum teste falhou → anote a mensagem de erro e chame um monitor em `#setup-ambiente`.
- Nada resolveu → sinalize para a equipe; a partir da Aula 02 existe uma imagem backup.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão da semana 📌

- Antes da Aula 02: responda o formulário de confirmação do ambiente (se ainda não fez).
- Brinque com o terminal sem medo.
- Se a VM "quebrar de vez", é só criar outra. Esse é o superpoder das máquinas virtuais!

**Dentro da VM você não quebra nada do seu computador.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

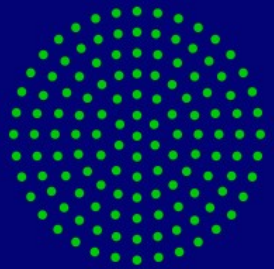
# O que levamos para a próxima aula

Na Aula 02, a tela preta deixa de ser assustadora e vira sua ferramenta de trabalho.

- DevOps é cultura + automação: Dev e Ops como um time só, do código à produção.
- O ciclo ♾️ e o jargão: CI, CD, pipeline, build e deploy.
- Ambiente validado: a VM Ubuntu é o laboratório das próximas 9 aulas.

**Toda automação roda em cima de um sistema operacional — e em 9 de cada 10 servidores do mundo, esse sistema é o Linux.**

REALIZAÇÃO:
Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 02

## Linux 1: História + Comandos básicos

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda de hoje

**Objetivo:** perder o medo da tela preta. Entender por que o Linux domina o mundo DevOps e sair navegando pelo terminal com confiança.

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: Por que o Linux domina o DevOps?
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa — primeiros passos no terminal
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

💬 **Interação: quem já abriu um terminal Linux alguma vez na vida? Responda sim ou não no chat.**

Apareceu uma palavra nova? Consulte o glossário do repositório (glossario.md).

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- DevOps é **cultura**: Dev + Ops num time só, automação e cultura blameless (sem culpados).
- O ciclo de vida DevOps é um loop (um ciclo que nunca para):
- planejar → codificar → buildar → testar → liberar → implantar → operar → monitorar.
- Todos validaram o ambiente: a VM Ubuntu é nosso laboratório.

**Toda essa automação roda em cima de um sistema operacional. Hoje conhecemos o rei deles: o Linux.** 🐧

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## Por que Linux domina o DevOps?

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Um pouco de história

**1991:** Linus Torvalds, um estudante finlandês, publica um **kernel** (o núcleo de um sistema operacional) "de hobby" na internet.

- E convida o mundo todo a colaborar pela internet.
- O Linux cresce como **software livre e de código aberto**.
- Código aberto: qualquer pessoa pode ler, modificar e distribuir.
- Hoje: na **imensa maioria dos servidores** e nos 500 supercomputadores mais rápidos.
- Em todo Android, nos carros, roteadores... e em toda a nuvem.

💬 **Interação: "Quantos dispositivos com Linux você acha que tem na sua casa? Chuta aí no chat!"**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# O mesmo coração, várias roupas

O **kernel** é o núcleo; as **distribuições** (distros) empacotam kernel + ferramentas: Ubuntu, Debian, Fedora, Alpine...

- Usamos o **Ubuntu**: popular em servidores e a distro mais amigável para começar.
- O coração é o mesmo (o kernel); mudam as roupas (as ferramentas empacotadas).

**Aprendeu Ubuntu, aprendeu Linux: o que vale aqui vale em qualquer distro.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Por que DevOps = Linux?

- Os servidores onde os sistemas rodam são Linux — quem opera precisa falar a língua deles.
- Containers (Aula 07!) **são** tecnologia do kernel Linux.
- Automação nasce no **terminal**: scripts, pipelines e ferramentas assumem um shell Linux.

**Dominar o terminal Linux é o alicerce de tudo o que vem depois no bootcamp.**

Container: uma "caixinha" isolada para rodar um programa — você vai conhecer de perto na Aula 07.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Anatomia do terminal (1/2)

A linha que fica esperando seus comandos chama-se **prompt**.

O programa que lê e executa o que você digita é o **shell** — no Ubuntu, o **bash**.

- `aluno` → usuário logado · `bootcamp` → nome da máquina
- `~` → diretório atual (`~` = sua pasta pessoal)
- `$` → usuário comum (`#` seria o superusuário root)

```
aluno@bootcamp:~$ ls -l /home
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Anatomia do terminal (2/2)

- `ls` → o **comando**
- `-l` → uma **opção** (modifica o comportamento)
- `/home` → o **argumento** (em quem o comando age)

```
aluno@bootcamp:~$ ls -l /home
```

REALIZAÇÃO:

Instituto MATERA

# Sem C:\ — uma única árvore

- Quem vem do Windows conhece `C:\`, `D:\` e pastas separadas por `\`.
- No Linux **não existem letras de unidade**: há uma única árvore, que começa na raiz `/`.
- As pastas são separadas por `/` (barra normal), nunca por `\`.
- Discos, pendrives e dispositivos são "encaixados" (montados) em algum ponto da árvore.
- **Maiúsculas ≠ minúsculas**: `Projeto` e `projeto` são pastas diferentes.

**Tudo começa na raiz `/`. Não existe C:, D: nem barra invertida.**

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

# Tudo é arquivo: o prédio (1/2)

No Linux, **tudo é arquivo**: documentos, programas, configurações, logs, dispositivos.

E tudo tem um lugar definido. Pense na árvore como um **prédio**, em que cada andar tem uma função:

```
/            ← a raiz: o terreno onde o prédio está
├── home/    ← apartamentos dos moradores (sua casa: /home/aluno)
├── root/    ← a cobertura: a casa do superusuário root
├── etc/     ← a administração: configurações do sistema
├── bin/ sbin/ ← a caixa de ferramentas: ls, cp, cat...
├── usr/     ← a loja de departamentos: programas instalados
├── opt/     ← o depósito: softwares "opcionais" de terceiros
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Tudo é arquivo: o prédio (2/2)

Continuando a visita pelos andares:

**Não precisa decorar: na Parte 2 você vai visitar o prédio pessoalmente.**

```
├── var/     ← o arquivo morto que só cresce: logs (/var/log)
├── tmp/     ← o lixão temporário: pode sumir ao reiniciar
├── dev/     ← a sala de máquinas: dispositivos (discos, terminais)
├── proc/ sys/ ← o painel de instrumentos: informações do kernel
├── boot/    ← os alicerces: o que o sistema precisa para ligar
├── lib/     ← bibliotecas compartilhadas pelos programas
├── mnt/ media/ ← as portas de entrada: discos externos e pendrives
└── srv/     ← dados de serviços servidos pela máquina (sites)
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# As pastas que você mais visita

Dos andares do prédio, estes são os que você mais vai frequentar em DevOps:

- `/home/aluno` — sua casa (`~`). É onde o projeto do bootcamp vai morar.
- `/etc` — toda configuração de serviço (rede, usuários, SSH, Docker...) vive aqui.
- `/var/log` — primeiro lugar para olhar quando algo quebra: os logs (registros).
- `/usr/bin` — onde ficam os programas que você instala com `apt` (o instalador do Ubuntu).
- `/tmp` — rascunhos descartáveis: nunca guarde nada importante aqui.

**Algo quebrou? Comece por `/var/log`. Quer configurar? Vá em `/etc`.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Caminhos e atalhos

- **Absoluto**: começa na raiz e vale de qualquer lugar: `/home/aluno/meu-projeto-devops`
- **Relativo**: parte de onde você está. Em `/home/aluno`, basta `meu-projeto-devops`
- `~` → sua pasta pessoal (`/home/aluno`) · `.` → a pasta atual
- `..` → a pasta "pai" (um nível acima): `cd ..` sobe um andar
- `/` → a raiz: `cd /` leva ao térreo

💬 **Interação: "Se eu estou em `/var/log` e digito `cd ..`, onde fui parar? E com `cd ~`?"**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Os comandos de hoje (1/2)

Dez comandos para começar. Cada um tem um mnemônico (uma frase em inglês que ajuda a lembrar).

- `pwd` — mostra onde você está (print working directory)
- `ls` — lista arquivos e pastas (list)
- `cd` — muda de pasta (change directory)
- `mkdir` — cria pasta (make directory)
- `touch` — cria arquivo vazio

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Os comandos de hoje (2/2)

- `cat` — mostra o conteúdo de um arquivo (concatenate)
- `cp` — copia (copy)
- `mv` — move ou renomeia (move)
- `rm` — remove (⚠️ sem lixeira!) (remove)
- `man` — abre o manual de qualquer comando (manual)

🧠 **Regra de ouro: na dúvida, `man comando`. O terminal vem com a documentação embutida.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Kit de sobrevivência (1/2)

Oito teclas que resolvem 90% dos sustos de quem está começando:

- `Tab` — **autocompleta** nomes de arquivos, pastas e comandos (2x `Tab` mostra as opções)
- Experimente: digite `cd meu-` + `Tab` e veja a mágica
- `↑ ↓` — navega pelos comandos que você já digitou: ninguém redigita comando
- `Ctrl + C` — **cancela** o comando atual. O terminal "travou"? É isto
- `Ctrl + L` (ou `clear`) — limpa a tela

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Kit de sobrevivência (2/2)

- `q` — sai de telas de leitura (`man`, `less`, o histórico do Git)
- Entrou no editor `vi` sem querer? `Esc`, depois `:q!` e Enter
- `Ctrl + R` — busca no histórico: comece a digitar e ele encontra
- `Ctrl + D` (ou `exit`) — encerra a sessão
- `comando --help` — ajuda resumida, quando o `man` for longo demais

**Tudo isso está na cola do terminal (recursos/cheat-sheet-terminal.md): imprima e deixe ao lado.** 📝

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Dois sustos clássicos

- **Apareceu um `>` sozinho e nada acontece.** Você abriu aspas e não fechou.
- O shell está esperando o resto: `Ctrl + C` e digite de novo.
- **O `~` não aparece (ou sai `~~`).** No teclado ABNT2 (o brasileiro), `~` é tecla morta.
- Aperte `~` e depois **Espaço**. E `|` é `Shift + \` (a tecla ao lado do `Z`).

**Susto no terminal não quebra nada: `Ctrl + C` e tente de novo.**

Veja o box de teclado no setup (setup/instalacao-ubuntu-vm.md).

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Primeiros passos no terminal

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Objetivo e onde

**Objetivo:** usar os comandos básicos para criar a estrutura do **seu projeto do bootcamp** — a pasta que vamos evoluir até a última aula.

**Onde:** no terminal conectado à VM via SSH (como no setup). A janela do VirtualBox também serve, mas sem copiar e colar.

💬 **Interação: conectou na VM e vê o prompt** `aluno@bootcamp:~$`**? Mande um 👍 no chat.**

Passo a passo completo em aulas/aula-02-linux-basico/pratica.md

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — Onde estou?

- Primeiro comando da vida de todo mundo no terminal: onde estou?
- `pwd` = print working directory: imprime o diretório em que você está agora.

```
pwd
```

Resultado esperado: /home/aluno (com o seu usuário). Essa é a sua casa, o ~.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 2 — Um tour pela árvore (1/3)

- `ls /` mostra o térreo: todas as outras pastas do sistema estão dentro de uma delas.
- O `~` é o atalho para a sua casa, de qualquer lugar.

```
ls /
cd /
pwd
cd ~
pwd
```

Resultado esperado: bin boot dev etc home lib … tmp usr var · / · /home/aluno

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 2 — Um tour pela árvore (2/3)

- Configurações do sistema vivem em `/etc`.
- O `|` (barra em pé, `Shift + \`) liga a saída de um comando ao próximo (próxima aula).
- `head -10` mostra só as 10 primeiras linhas: `/usr/bin` tem centenas de programas.

```
cat /etc/os-release
ls /usr/bin | head -10
```

Resultado esperado: NAME="Ubuntu" e VERSION="24.04..." · os 10 primeiros programas de /usr/bin

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 2 — Um tour pela árvore (3/3)

- `/var/log` é o primeiro lugar para olhar quando algo quebra.
- Depois, experimente o `..` (subir um nível).

```
ls /var/log
cd /var/log
cd ..
pwd
```

Resultado esperado: /var. Volte para casa com cd ~ antes do próximo passo.

# Passo 3 — Crie a pasta do projeto

- `mkdir` cria a pasta; `cd` entra nela; `pwd` confirma onde você está.
- Esta é a pasta que vamos evoluir até a última aula do bootcamp.
- Digite `cd meu-` e aperte `Tab`: o terminal completa o nome para você.

```
mkdir meu-projeto-devops
cd meu-projeto-devops
pwd
```

Resultado esperado: /home/aluno/meu-projeto-devops

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 4 — Crie a estrutura interna

- `mkdir` aceita vários nomes de uma vez: cria `scripts` e `app` num só comando.
- `touch` cria um arquivo vazio: o `README.txt` do projeto.
- `ls` lista o que você acabou de criar.

```
mkdir scripts app
touch README.txt
ls
```

Resultado esperado: app  README.txt  scripts

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — Escreva e leia um arquivo

- `echo` imprime um texto; o `>` "despeja" a saída do comando para dentro do arquivo.
- `cat` mostra o conteúdo do arquivo: a frase que você acabou de gravar.
- Feche as aspas! Se aparecer um `>` sozinho, é `Ctrl + C` e digite de novo.

```
echo "Projeto do Bootcamp DevOps Matera 2026" > README.txt
cat README.txt
```

Resultado esperado: o terminal mostra a frase que você gravou.

REALIZAÇÃO:

Instituto MATERA

# Passo 6 — Copie, renomeie, mova

- `cp origem destino` copia: agora existem dois arquivos.
- `mv origem destino` move (ou renomeia, se o destino for só um nome novo).
- `ls app/` confirma: o backup foi parar dentro da pasta `app/`.

```
cp README.txt README-backup.txt
mv README-backup.txt app/
ls app/
```

Resultado esperado: README-backup.txt agora está dentro de app/

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 7 — Remova com cuidado

- `rm` apaga de verdade: **no Linux não existe lixeira no terminal**.
- Antes de remover, respire e confira o caminho.
- Principalmente se um dia você tiver `sudo` (poderes de administrador) na frente. 😄

⚠️ `rm` **não pergunta e não tem volta. Confira o caminho antes do Enter.**

```
rm app/README-backup.txt
ls app/
```

Resultado esperado: app/ vazio de novo.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 8 — Explore o manual

- Navegue no manual com as setas e saia com a tecla `q`.
- Descubra: o que faz a opção `-a`? Depois teste com `ls -la`.
- Viu arquivos começando com `.`? São os ocultos: eles reaparecem na aula de Git!

```
man ls
ls -la
```

Resultado esperado: o manual do ls abre (q para sair); ls -la lista também os arquivos ocultos, que começam com .

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Deu errado? 😢

- `No such file or directory` → pasta errada: rode `pwd` e `ls` para se localizar
- `Permission denied` → saiu da sua pasta pessoal: por enquanto, fique dentro de `~`
- `command not found` → erro de digitação no comando: o terminal é implacável com isso
- O prompt não volta ou apareceu um `>` sozinho → `Ctrl + C` e digite de novo
- Abriu uma tela que não sai (`man`, `less`) → aperte `q`
- Entrou no editor `vi` sem querer → `Esc`, depois `:q!` e Enter

**Erro no terminal é normal. Leia a mensagem: ela quase sempre diz o que aconteceu.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus 🏆

Sem olhar as respostas: dentro de `meu-projeto-devops`, crie uma pasta `docs` com um arquivo `notas.txt`.

O arquivo deve conter a frase "Linux dominado!". Depois, mostre o conteúdo na tela.

- **Segundo desafio:** um único comando, com caminho absoluto, a partir da sua casa:
- mostre as 5 primeiras linhas de `/etc/passwd` (a lista de usuários do sistema).

💬 **Poste o print no chat! Dica do segundo desafio:** `head -5`**.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌

Hábito se faz com frequência: **5 minutos de terminal por dia** até a próxima aula.

- Todo dia, conecte na VM: vá até `/var/log`, volte com `cd ~`, liste com `ls -la`.
- Faça isso sem olhar o material.
- Use o `Tab` em todo nome que digitar — até virar reflexo.
- Leia o glossário e marque as palavras que ainda não fazem sentido: pergunte na comunidade.

**5 minutos por dia valem mais do que 1 hora na véspera.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que levamos para a próxima aula

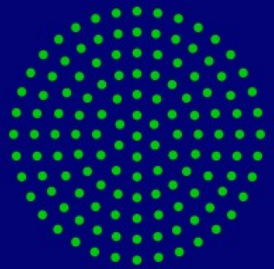
- Linux domina servidores, nuvem e containers: o terminal é o alicerce do DevOps.
- A árvore é um prédio que começa na raiz `/`: `/home`, `/etc`, `/var/log`, `/tmp`.
- Dez comandos (de `pwd` a `man`) e o kit de sobrevivência: `Tab`, `↑`, `Ctrl + C`, `q`.

**Hoje você aprendeu a andar pela casa. Na próxima aula, você vira o síndico: permissões, processos, programas e rede.**

Gancho para a Aula 03 — o dia a dia de quem administra servidores de verdade.

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 03

## Linux 2: Comandos intermediários

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda do encontro

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: Virando o síndico do servidor
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Hoje: permissões, processos, instalação de programas, pipes, rede e o editor `vi`.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- O Linux domina os servidores do mundo — e o terminal é a língua nativa dele.
- A árvore parte da raiz `/`: `/home`, `/etc`, `/var/log`, `/usr/bin`, `/tmp`
- Caminhos absolutos e relativos; `~` é a sua casa, `..` é a pasta de cima
- Navegar (`pwd`, `cd`, `ls`), criar (`mkdir`, `touch`, `echo >`), ler (`cat`)
- Organizar arquivos (`cp`, `mv`, `rm`)
- Nasceu o `meu-projeto-devops` — ele cresce a cada aula.

**Vocês já sabem andar pela casa. Hoje recebem as chaves do prédio: permissões, processos, instalação de programas e rede.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## Virando o síndico do servidor

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Usuários, root e sudo

- Todo arquivo e processo pertence a um **usuário**.
- O **root** é o superusuário: pode tudo (inclusive destruir tudo).
- `sudo` = "super user do": executa **um** comando como root.

**Com grandes poderes... você já sabe.** 🕷️

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Permissões: quem faz o quê

Três blocos de três letras: **dono**, **grupo** e **outros**. Um traço `-` significa 'não pode'.

- `rwx` — o **dono** pode ler (r), escrever (w) e executar (x)
- `r-x` — o **grupo** pode ler e executar
- `r--` — os **outros** só podem ler

```
-rwxr-xr--  1 aluno aluno  120 nov 10 19:40 script.sh
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# chmod e chown

- `chmod` muda permissões — ex.: `chmod +x script.sh` torna o arquivo executável
- Esse `+x` é essencial para a próxima aula (scripts)!
- `chown` muda o dono do arquivo

💬 **Interação: por que será que um arquivo de senha não deve ter `r` para 'outros'?**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Processos: o que roda agora?

**PID** é o número de identificação do processo — o `ps` mostra na segunda coluna.

- `ps aux` — fotografia de todos os processos
- `top` — monitor ao vivo (CPU, memória). Sai com `q`
- `kill <PID>` — encerra um processo pelo número dele

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Instalando com apt

No Ubuntu, programas vêm de **repositórios oficiais** via `apt` — o catálogo de programas do sistema.

- `update` baixa o catálogo atualizado; `install` instala o pacote
- Repare no `sudo`: instalar programas é tarefa de root

```
sudo apt update        # atualiza o catálogo
sudo apt install btop  # instala um pacote
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Segurança desde o dia zero 🔐

- Repositórios oficiais, sim; binários (programas prontos) de sites aleatórios, não
- Origem confiável e versões auditadas

**Esta é a primeira prática de DevSecOps que você aprende.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Superpoderes: pipes

O | (pipe) conecta a saída de um comando à entrada do próximo — é LEGO de comandos.

```
ps aux | grep bash          # filtra só o que contém "bash"
cat arquivo | wc -l          # conta linhas
ls -la | grep ".txt"        # só os .txt
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# grep e redirecionamento

- `grep` — procura texto (o comando mais usado da sua carreira!)
- `>` grava em arquivo (substitui o conteúdo)
- `>>` acrescenta ao final do arquivo

**O pipe liga comando a comando; o `>` liga comando a arquivo.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Redes em 4 conceitos

- **IP** — o endereço da casa (ex.: `192.168.0.10`)
- **Porta** — a porta da casa: cada serviço atende em uma (web: 80/443, SSH: 22)
- **DNS** — a agenda de contatos: traduz `matera.com` em IP
- **HTTP/HTTPS** — o idioma da conversa web (o S = conversa criptografada 🔒)
- **localhost** — o endereço da própria máquina (`127.0.0.1`)

**Falar com `localhost` é falar consigo mesmo.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Comandos de rede

- `ping` — o servidor responde?
- `curl` — faz requisições HTTP
- `ip a` — qual é o meu IP?

**Na prática de hoje vocês vão usar os três.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Micro aula: o editor vi

O `vi` existe em **todo** Linux do planeta — até em servidores sem mais nada instalado. A única ideia nova: ele tem **modos**.

- **Normal** (como abre) — controle remoto 🎮: cada tecla é um comando
- **Inserção** (`i`) — máquina de escrever ⌨️; aparece `-- INSERT --`
- **Linha de comando** (`:`) — menu 📋: salvar, sair, procurar

```
vi arquivo → [normal] → i → [inserção] → Esc → [normal] → :wq → Enter
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# vi: as teclas que salvam

- `i` — começa a escrever (modo de inserção)
- `Esc` — volta ao modo normal (aperte sem medo, quantas vezes quiser)
- `:wq` + Enter — salva e sai
- `:q!` + Enter — sai **sem** salvar: a saída de emergência 🚪
- `u` — desfaz

💬 **Interação: quem já ficou preso dentro de um editor no terminal? A partir de hoje:** `Esc`**,** `:q!`**, Enter.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Mão na massa

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Objetivo e onde

**Objetivo:** investigar o sistema, controlar permissões e falar com a internet — tudo dentro do `meu-projeto-devops`.

**Onde:** no terminal conectado à VM via SSH. A janela do VirtualBox também serve, mas sem copiar e colar.

**Prática 03 — Permissões, processos, pacotes e rede** 🔧

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — Instale sua primeira ferramenta

- `btop` é um `top` turbinado e colorido: veja CPU, memória, rede e processos
- Explore à vontade — e saia com `q`

```
sudo apt update
sudo apt install -y btop
btop
```

Resultado esperado: o btop abre em tela cheia com CPU, memória, rede e processos; sai com `q`.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 2 — Investigue processos com pipes

- Primeiro comando: você acabou de encadear **três** comandos!
- Segundo comando: quantos processos estão rodando agora?

```
ps aux | grep systemd | head -5

ps aux | wc -l
```

Resultado esperado: as 5 primeiras linhas de processos que contêm "systemd"; depois, o total de linhas.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 3 — Seu primeiro arquivo no vi

- 1. Aperte `i` (aparece `-- INSERT --` no rodapé)
- 2. Digite: `echo "ola, vi!"`
- 3. Aperte `Esc` (o `-- INSERT --` some)
- 4. Digite `:wq` e dê Enter

**O ritual do `vi`: devagar, uma tecla por vez.**

```
cd ~/meu-projeto-devops/scripts
vi teste.sh
```

Resultado esperado: você volta ao terminal e o arquivo `teste.sh` existe dentro de `scripts`.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Permissões na prática (1/2)

- `ls -l` mostra `-rw-rw-r--` — repare: **sem** `x`
- `./teste.sh` → `Permission denied`: sem `x`, o Linux se recusa a executar

```
cat teste.sh
ls -l teste.sh
./teste.sh
```

Resultado esperado: `Permission denied` — isso é o Linux funcionando, não um erro.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Permissões na prática (2/2)

- Depois do `chmod +x`: `-rwxrwxr-x` e o arquivo imprime `ola, vi!`
- Esse `x` é o que permitirá executar os scripts da próxima aula

```
chmod +x teste.sh
ls -l teste.sh
./teste.sh
```

Resultado esperado: `ola, vi!` na tela. Você escreveu o seu primeiro script!

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 4 — Descubra seu endereço na rede

- `ip a` lista as placas de rede (interfaces); o `grep inet` deixa só as linhas com IP
- `10.0.2.15` é o IP da sua VM na rede NAT do VirtualBox

```
ip a | grep inet
```

Resultado esperado: entre as linhas, algo como `inet 10.0.2.15/24`.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — Converse com a internet (1/2)

- `ping`: o servidor da Matera responde?
- Se responder, DNS e internet estão funcionando

```
ping -c 3 matera.com
```

Resultado esperado: `matera.com` responde aos três pings.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 5 — Converse com a internet (2/2)

- `curl -IL`: uma requisição HTTP de verdade, vendo só os cabeçalhos
- `302`/`307` = "vá para outro endereço" (redirecionamento); o `-L` segue até o `200`
- `200` = o famoso "OK" do HTTP. Repare no `https` 🔒: conversa criptografada

```
curl -IL https://www.matera.com | grep HTTP
```

Resultado esperado: linhas de status como `HTTP/2 302`, `HTTP/2 307` e `HTTP/2 200` — o que importa é a última.

REALIZAÇÃO:
Instituto MATERA

# Passo 6 — Grave um relatório do sistema

- `>` cria o arquivo; `>>` vai acrescentando ao final
- `date`, `uptime` e `df -h /`: data, tempo ligado e uso de disco

**Guarde esse encadeamento: ele vira um script na próxima aula!**

```
cd ~/meu-projeto-devops
echo "=== Relatorio do sistema ===" > relatorio.txt
date >> relatorio.txt
uptime >> relatorio.txt
df -h / | grep -v Filesystem >> relatorio.txt
cat relatorio.txt
```

Resultado esperado: um arquivo com data, tempo ligado e uso de disco.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢

- `Unable to locate package` → rode `sudo apt update` antes do `install`
- `Permission denied` no `apt` → faltou o `sudo`
- `ping: Name or service not known` → VM sem internet: confira o adaptador NAT
- `curl: command not found` → instale com `sudo apt install -y curl`
- Entrei no `vi` e não consigo sair → `Esc`, depois `:q!` e Enter (ou `:wq` para salvar)
- Coisas estranhas no `vi` → modo normal: `Esc`, `u` até voltar, depois `i`

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus 🏆

Usando pipes, conte quantos arquivos de configuração `.conf` existem em `/etc`.

**Poste o número no chat!**

```
ls /etc | grep ... | wc -l
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌

- 1. Abra o `btop` amanhã: qual processo mais consome memória na sua VM? Feche com `q`.
- 2. Com `grep`: quantas linhas do `/etc/passwd` contêm `nologin`? (Dica: `grep -c`.)
- 3. Rode `ls -l /etc | head -20` e leia as permissões de três arquivos em voz alta

**Em voz alta mesmo: "dono pode ler e escrever, grupo pode ler..."**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

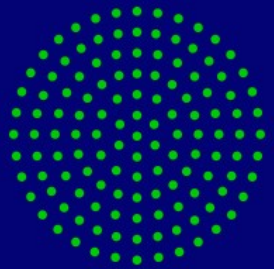
# O que levamos para a próxima aula

- Permissões (`rwx`, `chmod +x`) e o poder do `sudo`
- Pipes e `grep`: encadear comandos e filtrar o que importa
- O editor `vi`: `i`, `Esc`, `:wq`, `:q!` — e o seu primeiro `teste.sh` executável

**DevOps de verdade não digita a mesma sequência duas vezes — ele automatiza. Na próxima aula, seus comandos viram scripts.**

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 04

## Linux 3: Shell Script

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda de hoje

Objetivo da aula: transformar comandos em **automação**. Escrever, entender e executar os primeiros shell scripts.

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: Automatizando a vida com Shell Script
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa — seus primeiros scripts
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Shell script é a porta de entrada para tudo que vem depois: pipelines!**

Apareceu uma palavra nova? Consulte o glossário do repositório.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- Permissões (`chmod +x` — lembram do `x`? Hoje ele brilha!).
- Processos (`ps`, `top`), pacotes (`apt`) e pipes (`|`).
- Rede: IP é o endereço, porta é a porta da casa, DNS é a agenda, HTTP é o idioma.
- Fechamos gerando um `relatorio.txt` encadeando vários comandos... digitados um a um. 😓

**Gancho: e se aquele relatório se gerasse sozinho, com um único comando? Hoje seus comandos viram scripts.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## Automatizando com Shell Script

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que é um shell script?

Um **arquivo de texto com comandos** que o shell executa em sequência, de cima para baixo.

Tudo que você digita no terminal pode virar script. É a forma mais antiga — e ainda a mais usada — de automação em DevOps.

💡 **Toda pipeline de CI/CD que você verá na carreira é, no fundo, uma sequência de shell scripts executados por um robô.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Anatomia de um script

- `#!/bin/bash` — o **shebang**: diz qual interpretador executa o arquivo.
- `#` — comentários: explique o **porquê**, o computador ignora.
- `echo` imprime o texto na tela.

```
#!/bin/bash
# Meu primeiro script (isto é um comentário)

echo "Olá, bootcamp!"
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Como executar um script

- `chmod +x` dá permissão de execução — o `x` da Aula 03 preparou você!
- `./` significa 'o script que está nesta pasta'.
- Sem o `chmod +x`: `Permission denied`. Sem o `./`: `command not found`.

**Dois passos, sempre nesta ordem: dar permissão, depois executar.**

```
chmod +x script.sh
./script.sh
```

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Variáveis

- Uma variável é uma **caixinha com nome** onde você guarda um valor.
- `$NOME` lê o que está dentro da caixinha.
- `$(comando)` executa o comando e guarda a saída dele.

```
NOME="Matera"
echo "Bootcamp $NOME"        # usa a variável
DATA=$(date +%d/%m/%Y)        # guarda a SAÍDA de um comando
echo "Hoje é $DATA"
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Variáveis: as regras

- **Sem espaço** em volta do `=`: `NOME="Matera"` funciona, `NOME = "Matera"` quebra.
- Use `$VARIAVEL` para ler o valor.
- `$(comando)` captura a saída de um comando.
- Nomes em MAIÚSCULAS e descritivos ajudam a enxergar as variáveis no script.

💬 **Interação:** `NOME = "Matera"` **com espaços — o que acontece?**

# Condicionais: o script que decide

- `if` testa se o comando deu certo; `then` faz uma coisa, `else` faz outra, `fi` fecha.
- `ping -c 1` manda um único pacote para o site e espera a resposta.

**Um script que decide é um script que pode avisar você — é isso que o health check da Parte 2 faz.**

```
if ping -c 1 matera.com > /dev/null 2>&1; then
    echo "[OK] Internet"
else
    echo "[FALHA] Sem internet"
fi
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Código de saída e /dev/null

- Todo comando devolve um **código de saída**: 0 = sucesso, qualquer outro = falha.
- O `if` olha esse código — é assim que pipelines sabem se um passo falhou!
- `> /dev/null 2>&1` joga fora toda a saída do `ping`: só queremos saber se deu certo.
- O `/dev/null` é o 'buraco negro' do Linux: tudo que entra, some.

💬 **Interação: por que esconder a saída do ping dentro de um script?**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Loops: repetição sem sofrimento

- `for ... in lista; do ... done`: repete o bloco uma vez para cada item da lista.
- A cada volta, `$SERVICO` recebe o próximo nome.
- `systemctl` gerencia os serviços do sistema; `is-active` diz se o serviço está rodando.

```
for SERVICO in ssh cron systemd-resolved; do
    echo "Verificando $SERVICO..."
    systemctl is-active "$SERVICO"
done
```

REALIZAÇÃO:

Instituto MATERA

# Entrada do usuário e parâmetros

- `read -p` faz uma pergunta e guarda a resposta na variável.
- `$1` é o primeiro parâmetro: o que veio depois do nome do script na linha de comando.
- Parâmetro é o que torna um script útil para qualquer pessoa, não só para você.

```
read -p "Seu nome: " NOME     # pergunta interativa
echo "Bem-vindo, $NOME!"

# ou por parâmetro: ./script.sh Maria
echo "Olá, $1!"               # $1 = primeiro parâmetro
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Boas práticas desde o primeiro script

- Comece com `set -e`: o script **para no primeiro erro**. Falhar rápido é virtude.
- Nomes de variáveis MAIÚSCULOS e descritivos.
- **Nunca** coloque senhas ou tokens dentro do script — segredos ficam fora do código.

**Você verá onde os segredos ficam na Aula 10. Por enquanto: nunca dentro do script.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Seus primeiros scripts 

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Objetivo e onde

**Objetivo:** o relatório da Aula 03 vira um script de 5 linhas; depois, um **health check** que responde `[OK]` ou `[FALHA]`.

**Onde:** no terminal conectado à VM via SSH. A janela do VirtualBox também serve, mas sem copiar e colar.

- Health check (checagem de saúde): o script que verifica a saúde da VM.
- Os scripts usam só caracteres do teclado (sem emoji nem acento).
- Digite com calma — errar e corrigir faz parte — ou, se estiver via SSH, cole.
- Travou na digitação? No fim há como baixar as versões prontas.

**O health check fica no projeto até o fim do bootcamp.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — O relatório vira script (1/2)

- **vi em 1 minuto:** abre no modo normal; o mouse não funciona, use as setas.
- `i` entra na inserção (aparece `-- INSERT --`) · `Esc` volta ao normal.

```
cd ~/meu-projeto-devops/scripts
vi relatorio.sh
```

Resultado esperado: o vi abre vazio, pronto para receber o conteúdo do relatorio.sh.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — O relatório vira script (2/2)

- `:wq` + Enter salva e sai · `:q!` + Enter sai sem salvar · `dd` apaga a linha.
- Colar via SSH: `Esc`, `:set paste`, Enter, `i`, cole; depois `Esc`, `:wq`.

**Com o vi aberto, o próximo slide traz as cinco linhas do relatorio.sh.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — Conteúdo do relatorio.sh

- A primeira linha é o **shebang**: diz que o `bash` deve executar este arquivo.
- As outras quatro você já conhece da Aula 03.
- Salve e saia: `Esc`, `:wq`, Enter.

```
#!/bin/bash
echo "=== Relatorio do sistema ==="
date
uptime
df -h / | grep -v Filesystem
```

Resultado esperado: o vi fecha e você volta ao terminal, com o relatorio.sh salvo na pasta scripts.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 2 — Dê permissão e execute

- Quatro comandos, um só toque. **Isso é automação.**
- O `./` na frente significa 'o script que está nesta pasta'.
- O `x` do `chmod` é o da Aula 03.

💬 **Interação: deu certo? Cole no chat a linha do disco que apareceu para você.**

```
chmod +x relatorio.sh
./relatorio.sh
```

Resultado esperado: === Relatorio do sistema ===, a data, o uptime e a linha /dev/sda2 … 17% / (com os seus números).

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — O health check (versão mínima)

- Agora um script que **decide**: verifica internet e disco e responde `[OK]` ou `[FALHA]`.
- Vamos digitar em três blocos: cabeçalho, internet e disco.
- Lembre: `i` para inserir; se for colar via SSH, `:set paste` antes.

```
vi healthcheck.sh
```

Resultado esperado: o vi abre vazio, pronto para receber o conteúdo do healthcheck.sh.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Bloco 1: cabeçalho

- `set -e`: se qualquer comando falhar, o script para na hora.
- `$(hostname)` vira `bootcamp`; `$(date ...)` vira a data e a hora.
- Deixe uma linha em branco antes do próximo bloco.

```
#!/bin/bash
# healthcheck.sh - verifica a saude basica da maquina
set -e
echo "=== Health Check - $(hostname) - $(date '+%d/%m/%Y %H:%M') ==="
```

Resultado esperado: quatro linhas digitadas, sem acento; o cabeçalho será a primeira linha da saída.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 3 — Bloco 2: internet

- O mesmo `if` da Parte 1: se o `ping` responder, `[OK]`; senão, `[FALHA]`.
- `> /dev/null 2>&1` esconde a saída do `ping` — só queremos o veredito.
- Os quatro espaços antes do `echo` são só para ler melhor.

```
# 1) Internet: o ping respondeu?
if ping -c 1 ubuntu.com > /dev/null 2>&1; then
    echo "[OK] Internet"
else
    echo "[FALHA] Internet"
fi
```

Resultado esperado: seis linhas; o fi fecha o if. Linha em branco antes do próximo bloco.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Bloco 3: disco e fim

- `USO_DISCO` guarda só o número do percentual de uso do disco.
- `[ "$USO_DISCO" -lt 80 ]` testa se o número é menor que 80.

```
# 2) Disco: acima de 80% de uso, alerta
USO_DISCO=$(df / --output=pcent | tail -1 | tr -d ' %')
if [ "$USO_DISCO" -lt 80 ]; then
    echo "[OK] Disco: ${USO_DISCO}% usado"
else
    echo "[FALHA] Disco: ${USO_DISCO}% usado (acima de 80%!)"
fi

echo "=== Fim do health check ==="
```

Salve com Esc, :wq, Enter. Resultado: healthcheck.sh com 21 linhas, salvo na pasta scripts.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Comandos novos nesta prática (1/2)

- `set -e` → se qualquer comando falhar, o script para na hora (falhar rápido é virtude).
- `$(comando)` → executa o comando e usa a saída no lugar: `$(hostname)` vira `bootcamp`.
- `> /dev/null 2>&1` → joga fora o que o comando imprime; só importa se ele deu certo.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Comandos novos nesta prática (2/2)

- `df / --output=pcent` → mostra só a coluna de percentual de uso do disco.
- `tail -1` → pega só a última linha.
- `tr -d ' %'` → apaga os espaços e o `%`, sobrando só o número.
- `[ "$USO_DISCO" -lt 80 ]` → testa se o número é menor que (**less than**) 80.
- `${USO_DISCO}` → o mesmo que `$USO_DISCO`, com chaves para não grudar no texto ao lado.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 4 — Execute! 🚀

- Primeiro a permissão, depois a execução — sempre nesta ordem.
- Quatro linhas na saída: cabeçalho, internet, disco e fim.

💬 **Interação: tire um print e poste no chat!** 🎉

```
chmod +x healthcheck.sh
./healthcheck.sh
```

Resultado esperado: 4 linhas — cabeçalho com bootcamp e a hora, [OK] Internet, [OK] Disco: 17% usado e a linha de fim.

# Passo 5 — Grave o resultado em um log

- Automação de verdade guarda histórico.
- >> acrescenta no fim do arquivo (um > só sobrescreveria).
- Execute 2–3 vezes e veja o log crescer — cada execução com seu horário.

```
./healthcheck.sh >> ~/meu-projeto-devops/healthcheck.log
cat ~/meu-projeto-devops/healthcheck.log
```

Resultado esperado: o cat mostra os blocos de health check empilhados, um por execução, cada um com seu horário.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

# Travou na digitação? Baixe a versão pronta

- `curl -O` baixa o arquivo para a pasta atual, com o mesmo nome. Cole a URL via SSH.
- **Leia antes de executar** (`cat healthcheck.sh`): hábito de quem leva segurança a sério.
- Compare com o que você digitou, linha a linha: a diferença é o seu erro.

```
cd ~/meu-projeto-devops/scripts
curl -O https://raw.githubusercontent.com/matera-vrn/bootcamp-devops-2026/main/
aulas/aula-04-linux-shell-script/scripts/relatorio.sh
curl -O https://raw.githubusercontent.com/matera-vrn/bootcamp-devops-2026/main/
aulas/aula-04-linux-shell-script/scripts/healthcheck.sh
chmod +x relatorio.sh healthcheck.sh
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

# Deu errado? 😢

- `Permission denied` → faltou o `chmod +x`.
- `command not found` ao rodar → use `./healthcheck.sh` (com `./` na frente).
- `No such file or directory` → fora da pasta: `cd ~/meu-projeto-devops/scripts`.
- `syntax error near unexpected token` → `if`/`then`/`fi` incompleto; compare com o modelo.
- `integer expression expected` → linha `USO_DISCO=` errada; confira cada caractere.
- O terminal 'travou' → `Ctrl + C`; se foi o `ping`, confira se escreveu `-c 1`.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus 🏆

- **Memória:** 3ª verificação antes da última linha (`free -m` mostra a memória em MB).
- **Processos:** mais uma linha com quantos processos estão rodando.
- **Parâmetro:** o site do teste de internet vem em `$1`, com `ubuntu.com`

```
echo "[OK] Memoria disponivel: $(free -m | grep Mem | awk '{print $7}') MB"
ps aux | wc -l
SITE=${1:-ubuntu.com}
ping -c 1 "$SITE"
```

awk '{print $7}' pega a 7ª coluna do free, a available (disponível). O desafio 3 vira missão da semana.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌

- Toda vez que ligar a VM, rode o health check para o log e veja-o crescer com `tail -5`.
- Faça o desafio bônus 3 (site como parâmetro `$1`): um script que serve para qualquer um.
- Automatize uma chatice sua: um script de 3 linhas que cria a pasta do dia já conta.

```
~/meu-projeto-devops/scripts/healthcheck.sh >> ~/meu-projeto-devops/healthcheck.log
tail -5 ~/meu-projeto-devops/healthcheck.log
mkdir ~/rascunho-$(date +%d-%m)
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que levamos para a próxima aula

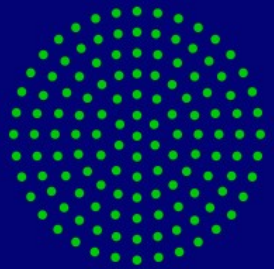
Seu script verifica a máquina em 2 segundos.

- Um script é um arquivo de texto com comandos: shebang, `chmod +x`, `./script.sh`.
- Variáveis, `if`, `for` e `$1` fazem o script **decidir** e servir para qualquer um.
- `set -e`, nomes descritivos e **nunca** segredos dentro do script.

**Gancho: encadear scripts de build, teste e deploy, executados por um robô a cada mudança de código. Isso tem nome: pipeline.**

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 05

## DevOps: Esteiras e Pipelines

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda de hoje

- 19:30 — Abertura, boas-vindas e avisos
- 19:35 — Parte 1: a esteira que mudou a indústria de software
- 20:20 — Intervalo ☕
- 20:35 — Parte 2: montando uma pipeline manual
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento pontual

**Hoje entramos no coração do DevOps: CI/CD e o conceito de esteira (pipeline).**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- Comandos viraram **scripts**: shebang, variáveis, `if`, `for`, códigos de saída.
- O `healthcheck.sh` verifica a VM em segundos e guarda histórico em log.
- `set -e`: em automação, **falhar rápido é uma virtude**.

**Um script automatiza uma tarefa. Uma esteira automatiza o caminho inteiro do código até produção.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## A esteira que mudou o software

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# A dor: o dia do deploy

Antes do CI/CD: meses de código acumulado, integração na véspera, deploy de madrugada, todo mundo rezando.

**Deploy** = colocar a versão nova no ar, em produção. Era um evento — e dava medo.

- Cada um trabalhou meses no seu canto; a **integração** só acontecia na véspera
- Quando quebrava (e quebrava), começava a **caça ao culpado**

# 💬 O que pode dar errado?

Cada resposta do chat é uma dor que o CI/CD nasceu para resolver.

💬 **Interação: "O que pode dar errado quando 10 pessoas juntam 3 meses de código de uma vez só? Joga no chat!"**

UPLOADS DE CONHECIMENTO
BOOT CAMP

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# CI — Integração Contínua (Continuous Integration)

Cada mudança de código entra no repositório várias vezes ao dia e, a cada entrada, um robô:

- 1. Pega o código
- 2. **Builda** (compila/empacota)
- 3. **Testa** automaticamente
- 4. Avisa imediatamente se algo quebrou

**Integrar pouco, integrar sempre.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Erro barato, erro caro

Erro descoberto em 5 minutos custa quase nada.

O mesmo erro descoberto em produção custa caro — em dinheiro e em sono. 😴

**Quanto mais cedo o erro aparece, mais barato ele é.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# CD — Entrega/Implantação Contínua

- **Entrega Contínua (Continuous Delivery)**
- Toda versão que passa na esteira fica **pronta para produção**
- O deploy vira uma decisão de negócio, a um clique
- **Implantação Contínua (Continuous Deployment)**
- Nem clique precisa — passou nos testes, vai pro ar

**Delivery: pronta para o ar, a um clique. Deployment: passou nos testes, foi.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# A esteira de fábrica 🏭

Uma fábrica moderna não monta um carro inteiro e só no final verifica se ele liga.

Cada estação da esteira **verifica e agrega valor**; se algo está errado, a esteira **para na hora**.

**O andon da Toyota — a corda que para a linha — inspirou o DevOps!**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Os stages da pipeline

No software, a esteira (pipeline) tem estações chamadas **stages**:

- Cada stage só roda se o anterior **passou**
- Quem decide se passou? Os **códigos de saída** da Aula 04!

```
[ Código ] → [ Build ] → [ Testes ]
    → [ Qualidade/Segurança ] → [ Deploy ]
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Verde ou vermelha

- Pipeline **verde** ✅ = pode confiar
- Pipeline **vermelha** ❌ = pare e conserte antes de seguir

**Vermelha não é vergonha: é a esteira te protegendo antes de ir para o ar.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Qualidade antes de ir para o ar

O que uma esteira verifica na vida real:

- **Build** — o código sequer compila/empacota?
- **Testes automatizados** — o que funcionava continua funcionando?
- **Análise estática (lint)** — o código segue os padrões?
- **Segurança (DevSecOps)** — há dependências vulneráveis? Segredos vazados no código?

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Ferramentas de mercado

- Jenkins, GitHub Actions, GitLab CI, Tekton, CircleCI...
- No bootcamp usaremos o **GitHub Actions** (Aula 10) — gratuito e acessível

**Os conceitos são os mesmos em todas. Aprenda o conceito e você troca de ferramenta em uma semana.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Hoje o robô é você 🤖

Na Parte 2 vamos construir uma esteira — build, teste e deploy — onde **cada stage é um script** e a esteira para se algum falhar.

- Deixe o terminal conectado à VM via SSH
- Vamos trabalhar em `~/meu-projeto-devops/scripts`

**Hoje o robô é você. Na Aula 10, o GitHub assume.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Montando uma pipeline manual

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Objetivo e onde

**Objetivo:** 3 stages — **build**, **teste** e **deploy** — cada stage é um script e a esteira **para no primeiro erro**.

**Onde:** no terminal conectado à VM via SSH, dentro de `~/meu-projeto-devops/scripts`.

- Nossa "aplicação" é uma página `index.html` — simples de propósito
- O build monta a página, o teste verifica, o deploy publica numa pasta que simula produção
- ⌨️ Quatro scripts curtos (28 linhas no total). Digite com calma ou cole bloco a bloco

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — Stage de build

- Entre na pasta `scripts` e abra o primeiro script no `vi`
- Via SSH, digite `:set paste` no vi antes de colar — assim o texto não se desalinha
- Vamos escrever o conteúdo do `build.sh` no próximo slide (6 linhas)

```
cd ~/meu-projeto-devops/scripts
vi build.sh
```

Resultado esperado: o vi aberto em um arquivo build.sh vazio, dentro de ~/meu-projeto-devops/scripts.

REALIZAÇÃO:
Instituto MATERA

# Passo 1 — Conteúdo do build.sh

- A linha do `echo "<h1>...` é **uma só**, comprida — não aperte Enter no meio dela
- O "build" aqui gera a página com a data e hora — numa aplicação real, seria compilar

```
#!/bin/bash
set -e
echo "[BUILD] Montando a aplicacao..."
mkdir -p ~/meu-projeto-devops/app
echo "<h1>Bootcamp DevOps Matera 2026</h1><p>Build em $(date '+%d/%m/%Y %H:%M:
%S')</p>" > ~/meu-projeto-devops/app/index.html
echo "[BUILD] OK - app/index.html gerado."
```

Salve com :wq. Ainda não execute — primeiro escrevemos os quatro scripts.

# Passo 2 — Stage de teste (1/2)

- Abra com `vi test.sh` e digite as 9 linhas acima
- Salve com :wq — a explicação vem no próximo slide

```
#!/bin/bash
set -e
echo "[TEST] Verificando a aplicacao..."
if grep -q "Bootcamp DevOps" ~/meu-projeto-devops/app/index.html; then
    echo "[TEST] OK - conteudo esperado encontrado."
else
    echo "[TEST] FALHOU - conteudo esperado nao encontrado!"
    exit 1
fi
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

# Passo 2 — Stage de teste (2/2)

- O `if` pergunta ao `grep -q` se o texto está lá: achou → OK; não achou → FALHOU
- `exit 1` é o coração da esteira: código de saída diferente de zero significa **"falhei"**
- Quem chamou o script fica sabendo — é isso que faz a esteira parar

Salve com :wq. O teste procura o texto "Bootcamp DevOps" na página gerada pelo build.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

# Passo 3 — Stage de deploy

- 1ª linha é o comando no terminal; as 6 seguintes são o arquivo
- Copia a página gerada para `~/producao` — nossa "produção" de mentira

```
vi deploy.sh
#!/bin/bash
set -e
echo "[DEPLOY] Publicando em 'producao'..."
mkdir -p ~/producao
cp ~/meu-projeto-devops/app/index.html ~/producao/
echo "[DEPLOY] OK - publicado em ~/producao/index.html"
```

Salve com :wq. mkdir -p garante a pasta antes do cp. Três stages prontos; falta a esteira que liga tudo.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 4 — A esteira liga tudo

- Um script chamando os outros três, na ordem: build → test → deploy
- Com `set -e`, se um stage devolver erro, a esteira **para ali**

```
vi pipeline.sh
#!/bin/bash
set -e
echo "===== PIPELINE INICIADA - $(date '+%H:%M:%S') ====="
./build.sh
./test.sh
./deploy.sh
echo "===== PIPELINE VERDE - aplicacao em producao! ====="
```

Salve com :wq. Esta é a pipeline: 7 linhas e nenhuma ferramenta além do bash.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Comandos novos

- `mkdir -p pasta` — cria a pasta se não existir (e não reclama se já existir)
- `grep -q "texto" arquivo` — procura o texto **em silêncio** (quiet), sem imprimir nada
- Só responde "achei" (sucesso) ou "não achei" (erro) — perfeito para um `if`
- `exit 1` — encerra o script avisando que falhou (`0` = sucesso; outro número = falha)
- `./build.sh` dentro de outro script — um script pode chamar outros e encadear os stages

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 5 — Execute a esteira!

- `chmod +x` dá permissão de execução aos 4 scripts de uma vez
- `./pipeline.sh` chama os três stages na ordem
- Depois, confira a "produção" com o `cat`

```
chmod +x build.sh test.sh deploy.sh pipeline.sh
./pipeline.sh
cat ~/producao/index.html
```

Resultado esperado: termina com ===== PIPELINE VERDE - aplicacao em producao! =====

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — Pipeline verde ✅

- Cada stage imprime o seu prefixo: `[BUILD]`, `[TEST]`, `[DEPLOY]`
- A hora da sua pipeline será diferente — o resto deve ser igual

```
===== PIPELINE INICIADA - 20:50:12 =====
[BUILD] Montando a aplicacao...
[BUILD] OK - app/index.html gerado.
[TEST] Verificando a aplicacao...
[TEST] OK - conteudo esperado encontrado.
[DEPLOY] Publicando em 'producao'...
[DEPLOY] OK - publicado em ~/producao/index.html
===== PIPELINE VERDE - aplicacao em producao! =====
```

Resultado esperado: cat ~/producao/index.html mostra o <h1> com a data e hora do build.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Quebre a pipeline 💥 (1/2)

Cultura **blameless** (sem culpados) na prática: vamos provocar uma falha e ver a esteira nos proteger.

- `vi build.sh`: troque `Bootcamp DevOps Matera 2026` por `Ola mundo` e salve
- Rode a pipeline de novo e observe **onde** ela para — os stages seguintes não rodam

```
./pipeline.sh
```

Resultado esperado: [TEST] FALHOU e nenhum [DEPLOY].

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Quebre a pipeline 💥 (2/2)

- 🔎 Confirme: `cat ~/producao/index.html` ainda mostra a versão boa anterior
- Depois desfaça a mudança no `build.sh`, rode de novo e deixe tudo verde 💚

**O deploy nunca executou. A versão quebrada não chegou em produção. Isso é qualidade antes de ir para o ar!**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Travou na digitação?

- Os quatro scripts estão no repositório do bootcamp (pratica.md, "Travou na digitação?")
- Via SSH, baixe com `curl -O` para `~/meu-projeto-devops/scripts`, um comando por script
- **Leia antes de executar** (`cat build.sh`) e dê `chmod +x` nos quatro

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢

- `Permission denied` → faltou `chmod +x` em algum dos 4 scripts
- `No such file or directory` ao rodar `./pipeline.sh` → rode de dentro da pasta `scripts`
- `grep: ... No such file or directory` → o `test.sh` rodou antes do `build.sh`
- Pipeline não para no erro → confira o `set -e` no `pipeline.sh` e o `exit 1` no `test.sh`
- A página saiu sem a data → a linha do `echo "<h1>...` ficou quebrada em duas no `vi`

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Desafio bônus 🏆

- **1. Smoke test:** 4º stage — o `healthcheck.sh` da Aula 04 rodando **depois** do deploy
- Na vida real, checar se produção segue saudável após a mudança se chama "smoke test"
- **2. Teste mais claro:** se o `index.html` não existir, o `grep` reclama em inglês
- Adicione antes do `grep` um teste só para isso (`-f` = "o arquivo existe?"; `!` = "não"):

```
if [ ! -f ~/meu-projeto-devops/app/index.html ]; then echo "[TEST] FALHOU -
index.html nao existe!"; exit 1; fi
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌

- 1. Rode a pipeline mais duas vezes esta semana e provoque uma falha **diferente** da aula
- Apague a página (`rm ~/meu-projeto-devops/app/index.html`) e rode só o `./test.sh`
- Ele reprova mesmo sem o arquivo? Depois faça o desafio bônus 2 para a mensagem ficar sua
- 2. Faça o desafio bônus 1 (health check como 4º stage)
- Na próxima aula você vai versionar tudo isso — é bom que esteja do seu jeito

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

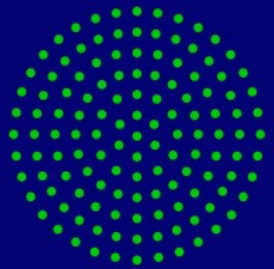
UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que levamos para a próxima aula

- CI: integrar pouco, integrar sempre — build e testes a cada mudança
- CD: toda versão verde fica pronta para produção
- A esteira para no primeiro erro — e é o `exit 1` quem decide

**Gancho: a esteira roda em cima de qual versão do código? Sem controle de versão não existe CI/CD. Próxima aula: Git.**



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 06

## Git: onde o código vive

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Agenda de hoje

Objetivo: entender o que é controle de versão, por que o Git é a **fonte única da verdade** do DevOps e dominar o fluxo básico.

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: Controle de versão e a fonte única da verdade
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa — o `meu-projeto-devops` vira repositório Git
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Ao final de hoje:** `init`, `add`, `commit`, `branch` **e** `merge` **na ponta dos dedos.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Revisão da aula anterior

- CI = integrar pouco e sempre; CD = toda versão pronta (ou indo) para produção.
- Vocês montaram uma pipeline com stages de build → teste → deploy.
- A esteira **para no primeiro erro** — e vocês viram um teste barrar uma versão quebrada.

**Mas a esteira roda em cima de qual versão do código? Se dois colegas mudam o mesmo arquivo, quem vence?**

Hoje conhecemos a fonte única da verdade: o Git.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## A fonte única da verdade

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# A dor que o Git cura

- Sem controle de versão: ninguém sabe qual é a versão atual.
- Ninguém sabe quem mudou o quê, nem como voltar atrás.
- Em equipe, vira caos multiplicado.

**Controle de versão = registrar cada mudança para saber o que é atual, quem mudou e como voltar.**

```
sistema_final.zip
sistema_final_v2.zip
sistema_final_AGORA_VAI.zip  😅
```

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que é o Git

Criado em 2005 por Linus Torvalds (o mesmo do Linux!) para organizar o desenvolvimento do kernel — o núcleo do Linux.

- Registra **cada mudança**, de **cada arquivo**, feita por **cada pessoa**, para sempre.
- Nasceu para um projeto gigante e distribuído: milhares de pessoas no mesmo código.
- Hoje é o padrão em praticamente toda equipe de software.

**Git = a memória completa do seu projeto.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que o Git guarda

- **Histórico completo:** quem, quando, o quê e por quê (a mensagem de commit!).
- **Volta no tempo:** qualquer versão pode ser restaurada.
- **Em paralelo:** cada um na sua linha do tempo (branch), juntando depois (merge).

**Commit, branch e merge: daqui a pouco, a definição exata de cada um — e o desenho.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Git no DevOps

No DevOps, o repositório Git (a pasta do projeto com todo o seu histórico) não guarda só código da aplicação:

- Scripts de automação ✔️ (os seus da Aula 04–05!)
- Definição da pipeline ✔️ (Aula 10)
- Infraestrutura como código ✔️
- Documentação ✔️

**É do Git que a esteira parte; é o Git que conta a história do sistema.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Se não está no Git, não existe

O repositório é a **fonte única da verdade**: o lugar onde todo mundo concorda sobre qual é a versão atual.

- Não é 'a versão que está na máquina do fulano'.
- Não é o zip no e-mail nem a pasta compartilhada.
- Este próprio bootcamp vive em um repositório Git!

**Se não está no Git, não existe.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Os três estados de um arquivo

- **Bancada** (working directory): onde você edita os arquivos.
- **Caixa** (staging area): o que você escolheu enviar nesta versão.
- **Cofre** (repositório): o histórico, selado para sempre.

```
[ Working Directory ]   (sua bancada)
          |  git add
          v
[ Staging Area ]        (a caixa a enviar)
          |  git commit
          v
[ Repositório ]         (o cofre com histórico)
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Comandos do dia a dia

- `git init` — transforma uma pasta em repositório.
- `git status` — o comando que você mais vai usar: o que mudou?
- `git add arquivo` — coloca a mudança na "caixa" (staging).
- `git commit -m "mensagem"` — sela a caixa no histórico, com autor e data.

**Ritual:** `git status` → `git add` → `git commit`**. Sempre nessa ordem.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Comandos: histórico e branches

- `git log` — mostra a linha do tempo.
- `git checkout` — alterna entre ramificações (branches).
- `git merge` — traz o conteúdo da branch informada para a branch atual.

**`log` para olhar o passado; `checkout` e `merge` para navegar entre linhas do tempo.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Boas mensagens de commit

🧠 Boas mensagens de commit dizem **o quê** e **o porquê**.

- `"adiciona teste de disco ao healthcheck"` ✔️
- `"ajustes"` ✖️

**A mensagem é para o você de daqui a seis meses — e para o colega que nunca viu esse código.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Commit: a definição

**Commit** = registro **permanente** do projeto num certo momento.

- **Hash** único (`e0b70f5`): a impressão digital do conteúdo.
- **Autor, data e mensagem**: quem, quando e por quê.
- Aponta para o **pai** (o commit anterior): é a linha do tempo.
- Não muda nunca: corrigir = fazer um commit novo.

```
A---B---C     cada letra = um commit
              C aponta para B, que aponta para A
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Branch e HEAD: a definição

- **Branch** = um **nome que aponta para um commit**: o mais recente da linha.
- Não copia arquivos: é um arquivo de texto com um hash (`.git/refs/heads/main`).
- Commit feito numa branch → o nome dela **avança sozinho**.
- **HEAD** = onde você está: aponta para a branch em uso (`HEAD -> main`).
- O HEAD define os arquivos da pasta e **qual branch avança** no próximo commit.

**Commit é a página. Branch é o marcador. HEAD é o dedo.**

REALIZAÇÃO:
Instituto MATERA

# Branches em desenho (1/2)

- Criar branch **não cria commit**: só um nome a mais, no mesmo lugar.
- **Fast-forward**: a `main` não andou, então o Git só avança o nome.

```
git switch -c ideia     A---B   <- main
                                <- ideia <- HEAD

commit na ideia         A---B   <- main
                             \
                              C   <- ideia <- HEAD

git switch main         A---B---C   <- main <- HEAD
git merge ideia                     <- ideia
```

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Branches em desenho (2/2)

- Se a `main` também andou, o merge cria **M**, com **dois pais**.
- Mesma linha mudada dos dois lados? **Conflito**: você decide.
- No terminal: `git log --oneline --graph --all`.

**Fast-forward avança um nome. Merge commit junta duas histórias.**

```
as duas andaram:          merge commit:
A---B---D  <- main        A---B---D---M  <- main <- HEAD
     \                         \     /
      C    <- ideia              C---'     <- ideia
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Branches: o fluxo de trabalho

- `main` — a linha oficial, sempre funcional.
- `git checkout -b minha-ideia` (ou `git switch -c`) — cria a branch e move o HEAD.
- Experimente sem medo, longe da `main`.
- `git checkout main` + `git merge minha-ideia` — aprovada? Junta na linha oficial.

**A `main` nunca quebra: toda experiência acontece numa branch e só entra depois de aprovada.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Onde o histórico mora

Lembram do `ls -la` da Aula 02, que mostrou arquivos ocultos?

💬 **Interação: rodem `ls -la` na pasta do projeto depois do `git init` e me digam o que apareceu!**

Resposta: a pasta `.git`, onde o histórico mora.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Versionando o projeto

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Objetivo e onde

**Objetivo:** transformar o `meu-projeto-devops` em um repositório Git, fazer os primeiros commits, criar uma branch e fazer um merge.

- **Onde:** no terminal conectado à VM via SSH (como no setup).
- A janela do VirtualBox também serve, mas sem copiar e colar.
- Siga o `pratica.md` da aula 06 no repositório do bootcamp.

**Ao final: histórico, branch e merge de verdade no seu projeto.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — Apresente-se ao Git

- O Git já vem no Ubuntu Server; se der `command not found`: `sudo apt install -y git`.
- O Git assina cada commit com seu nome. Configure uma vez só.
- 💡 O e-mail fica **público** no GitHub (Aula 09). Prefere não expor? Use o noreply de lá.

```
git --version
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
git config --global init.defaultBranch main
```

Resultado esperado: `git version 2.43.x`; os `git config` não mostram nada — silêncio é sucesso.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 2 — Nasce um repositório

- `git init` transforma a pasta em repositório.
- O `ls -la` mostra a pasta oculta `.git` — o cofre do histórico.

```
cd ~/meu-projeto-devops
git init
ls -la
```

Resultado esperado: `Initialized empty Git repository...` e, no `ls -la`, a pasta oculta `.git`.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Status, o seu melhor amigo

- Arquivos em vermelho como untracked (não rastreados): o Git os vê, mas não os protege.
- Esse é o comando que você mais vai usar daqui para a frente.

```
git status
```

Resultado esperado: arquivos em vermelho como untracked.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 4 — Primeiro commit! 🎉 (1/3)

- `rm` tira o `scripts/teste.sh` (Aula 03); `vi .gitignore` cria a lista do que ignorar.
- Duas arrumações antes do primeiro commit. O conteúdo do `.gitignore` vem no próximo slide.

```
rm scripts/teste.sh
vi .gitignore
```

Lembrete do vi: `i` para inserir, `Esc`, `:wq` para salvar.

REALIZAÇÃO:
Instituto MATERA

# Passo 4 — Primeiro commit! 🎉 (2/3)

- Estas 4 linhas são o **conteúdo** do `.gitignore`: gerados não se versionam.
- Salve no vi (`Esc`, `:wq`) e rode `git status` de novo.

```
# arquivos gerados - nao versionar
*.log
relatorio.txt
app/index.html
```

Resultado esperado: `healthcheck.log`, `relatorio.txt` e `app/index.html` sumiram da lista do `git status`.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 4 — Primeiro commit! 🎉 (3/3)

- `git add .` coloca tudo que mudou na caixa.
- `git commit -m` sela a caixa com seu nome, data e mensagem.
- Essa versão agora existe para sempre.

```
git add .
git commit -m "primeiro commit: scripts de healthcheck e pipeline"
git status
git log
```

Resultado esperado: `nothing to commit, working tree clean` e um `git log` com seu nome, data e mensagem.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 5 — Mude algo e veja o Git perceber

- O `git diff` mostra exatamente **a linha** que mudou.
- Depois de conferir, sele a mudança: `add` e `commit`.

```
echo "Projeto versionado com Git!" >> README.txt
git status
git diff
git add README.txt
git commit -m "atualiza README com nota sobre versionamento"
git log --oneline
```

Resultado esperado: duas linhas no `git log --oneline` — sua linha do tempo cresceu.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Branch: experimente sem medo (1/2)

- `checkout -b` cria a branch e já entra nela.
- A linha do `echo` é **conteúdo** do script: vai no final, antes da última linha `echo`.

```
git checkout -b melhoria-healthcheck
vi scripts/healthcheck.sh

echo "[OK] Tempo ligado: $(uptime -p)"
```

Lembrete do vi: `i` para inserir, `Esc`, `:wq` para salvar.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Branch: experimente sem medo (2/2)

- Quem avança é a branch do HEAD: a `melhoria-healthcheck` foi para o `C`.
- A `main` continua parada no `B` — a prova vem no Passo 7.

```
git add scripts/healthcheck.sh
git commit -m "adiciona tempo ligado ao healthcheck"

A---B   <- main
     \
      C   <- melhoria-healthcheck <- HEAD
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 7 — Compare os universos paralelos

- Na `main`, sua mudança **não existe**! Ela vive só na branch.
- Depois do `merge`, a mudança chegou à `main`.
- Em miniatura, é assim que times inteiros trabalham.

```
git checkout main
cat scripts/healthcheck.sh | tail -3
git merge melhoria-healthcheck
cat scripts/healthcheck.sh | tail -3
git log --oneline
```

Resultado esperado: a mudança chegou à `main` — ideia testada, aprovada e integrada; o log mostra o novo commit.

REALIZAÇÃO:
Instituto MATERA

# Passo 7 — O que o merge fez

- **Fast-forward**: a `main` não andou, então o Git só avançou o nome até o `C`.
- No log: `(HEAD -> main, melhoria-healthcheck)` — dois nomes, o mesmo commit.
- Branch cumpriu o papel? `git branch -d melhoria-healthcheck`.

```
antes do merge:   A---B      <- main <- HEAD
                       \
                        C    <- melhoria-healthcheck

depois:           A---B---C  <- main <- HEAD
                             <- melhoria-healthcheck
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢

- `Please tell me who you are` → faltou o `git config` do Passo 1
- `not a git repository` → você não está dentro de `~/meu-projeto-devops`
- `nothing to commit` → faltou o `git add` antes do commit, ou nada mudou
- Abriu um editor no merge/commit → é o `vi` pedindo a mensagem: `Esc`, `:wq` e Enter
- `git log` abriu uma tela que não sai → visualizador de páginas: setas navegam, `q` sai

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus 🏆

- Crie uma branch `experimento`.
- Faça um commit **quebrando** o healthcheck de propósito.
- Volte para a `main`... e simplesmente **apague a branch**: `git branch -D experimento`.

**Sinta o poder: o erro nunca tocou a linha oficial.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌

- Faça pelo menos **dois commits reais** até a próxima aula.
- Ideias: melhorar um comentário, acrescentar linha ao `README.txt`, ajustar o health check.
- Mensagens que dizem o que e por quê.
- Antes de cada commit: `git status` e `git diff` — e só depois `git add`.
- Veja sua história crescer: `git log --oneline`.

**Esse ritual é o que separa quem usa Git de quem sofre com Git.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que levamos para a próxima aula

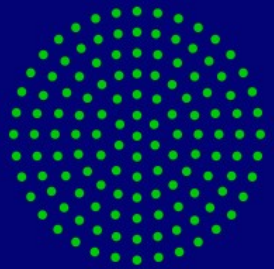
- Git é a **fonte única da verdade**: se não está no Git, não existe.
- Bancada → caixa (`add`) → cofre (`commit`). Ritual: `status`, `diff`, `add`, `commit`.
- Branch para experimentar sem medo; `merge` para integrar o que foi aprovado.

**Seu projeto agora tem histórico e versão. Mas quando ele rodar na máquina de outra pessoa... será que funciona?** 🐳

Aula 07 (amanhã, 12/11): 'na minha máquina funciona' — containers e Docker.

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 07

## Docker 1: Overview + Comandos

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda de hoje

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: O poder dos containers
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa — primeiro container e primeira imagem
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Objetivo: entender o que são containers, a diferença real para a máquina virtual e rodar os primeiros containers com Docker.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- Git = fonte única da verdade: `add` → `commit` → histórico para sempre
- Branches para experimentar, merge para integrar
- O `meu-projeto-devops` agora tem linha do tempo própria

**O código está versionado... mas rodar na máquina do colega ainda é loteria. Hoje empacotamos o ambiente inteiro com o código!** 🐳

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## O poder dos containers

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# "Na minha máquina funciona"

O app funciona na máquina do dev e quebra no servidor. Por quê?

Porque o **ambiente** é diferente.

- Outra versão de linguagem
- Outra biblioteca
- Outro sistema operacional

**O código viajou, mas o ambiente ficou para trás.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Antes do contêiner padrão 

Antes dos anos 1950, cada carga era embarcada do seu jeito: sacas, barris, caixas.

- Lento
- Caro
- Quebrava tudo

**Aí veio o contêiner padronizado: não importa o que tem dentro, todo navio, trem, caminhão e guindaste sabe transportá-lo.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Docker fez isso com software

Sua aplicação + todas as dependências dentro de uma "caixa padrão".

- Roda **igual** na sua VM
- Roda **igual** no servidor da empresa
- Roda **igual** na nuvem

**A caixa padrão do software: o container roda igual em qualquer lugar.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Container vs. Máquina Virtual

- **Carrega:** VM = sistema operacional inteiro + app · Container = só app + dependências
- **Tamanho:** VM = gigabytes · Container = megabytes
- **Inicia em:** VM = minutos · Container = segundos
- **Isolamento:** VM = hardware virtual completo · Container = processos isolados (kernel)

💬 **Interação: vocês construíram uma VM no setup — quanto tempo levou até ela ficar pronta?**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Casa vs. apartamento 🏠

- **VM** = casa completa: fundação, encanamento, tudo próprio
- Vocês construíram uma casa dessas no setup!
- **Container** = apartamento: isolado, com fechadura própria, mas na estrutura do prédio
- A estrutura do prédio = o **kernel do Linux** (o núcleo do sistema, da Aula 02)

💡 **Lembra da Aula 02? Containers são tecnologia do kernel Linux — por isso o Linux veio primeiro no bootcamp.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Os três conceitos do Docker

- **Dockerfile** = a receita 📝: texto com o passo a passo para montar o ambiente
- **Imagem** = o bolo pronto congelado 🎂: pacote imutável construído a partir da receita
- **Container** = a fatia servida 🍰: uma imagem em execução (pode haver várias!)

**Receita → bolo → fatia: Dockerfile → imagem → container.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Docker Hub: a padaria pública

Um **registro** (depósito de imagens na internet) com milhares de imagens prontas: `ubuntu`, `nginx`, `python`...

- Você não precisa assar todo bolo do zero: muita coisa já vem pronta
- `docker run` baixa a imagem de lá automaticamente (vamos ver na prática)

**Milhares de imagens prontas para você começar: a padaria pública do Docker.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Anatomia de um Dockerfile

- `FROM` — comece da imagem oficial do nginx (um servidor web)
- `COPY` — copie meu app para dentro
- `EXPOSE` — documente a porta usada

**Três linhas de receita: de onde parto, o que coloco dentro, qual porta uso.**

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Comandos essenciais (1/2)

- `docker run <imagem>` — baixa (se preciso) e executa um container
- `docker ps` / `docker ps -a` — containers rodando / todos
- `docker build -t nome .` — constrói uma imagem a partir do Dockerfile

**run executa, ps lista, build constrói.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Comandos essenciais (2/2)

- `docker stop` / `docker rm` — para / remove um container
- `docker images` — lista as imagens locais
- Ciclo que vamos repetir hoje: `run` → `ps` → `stop` → `rm`

**Parar não é remover: o container parado continua na lista do `docker ps -a`.**

REALIZAÇÃO:
Instituto MATERA

# Publicando portas: -p 8080:80

`-p 8080:80` publica portas: porta 8080 da VM → porta 80 do container.

- 8080 = porta **externa** (da VM)
- 80 = porta **interna** (do container)
- Quem bate na VM na 8080 cai no container na 80

🔢 **Lembra de IP e portas da Aula 03? O `-p externa:interna` é a ponte entre o mundo e o container.**

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Mão na massa

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Objetivo e onde

- Instalar o Docker na VM
- Rodar o primeiro container
- Containerizar a página do seu projeto com um `Dockerfile`
- **Onde:** no terminal conectado à VM via SSH (como no setup)
- A janela do VirtualBox também serve, mas sem copiar e colar

**Prática 07 — Primeiro container e primeira imagem** 🐳

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — Instale o Docker

- Instalamos do repositório oficial do Ubuntu: simples e confiável
- `docker-buildx` evita o aviso de construtor antigo ("legacy builder") no `docker build`
- A instalação baixa alguns pacotes: pode levar um ou dois minutos

```
sudo apt update
sudo apt install -y docker.io docker-buildx
docker --version
```

Resultado esperado: `Docker version 2x.x.x, build ...`

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — Docker sem sudo

- Adicione seu usuário ao grupo `docker` e **saia e entre de novo** para valer
- O `ssh aluno@localhost -p 2222` é no terminal do seu computador
- 🔐 No grupo `docker` você faz quase tudo que o root faz — na VM de estudo, tudo bem

```
sudo usermod -aG docker $USER
exit
ssh aluno@localhost -p 2222
groups
```

Resultado esperado: a palavra `docker` aparece na lista do `groups`.

REALIZAÇÃO:

Instituto MATERA

# Passo 2 — O "hello world" dos containers

- O Docker **baixou** a imagem do Docker Hub e **executou** um container
- O container imprimiu a mensagem e terminou

```
docker run hello-world
docker ps -a
```

Resultado esperado: `Hello from Docker!`

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 3 — Um servidor web em UM comando

- `-d` = em segundo plano; `--name meu-nginx` = nome do container
- `-p 8080:80` liga a porta **8080 da VM** à porta **80 do container**
- Teste com o `curl` da Aula 03; depois pare e remova

```
docker run -d --name meu-nginx -p 8080:80 nginx:alpine
curl http://localhost:8080
docker stop meu-nginx
docker rm meu-nginx
```

Resultado esperado: o HTML de boas-vindas do nginx — um servidor web em segundos, sem instalar nginx na VM!

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 4 — A receita: seu primeiro Dockerfile

- Vamos containerizar a página que a sua pipeline da Aula 05 gera
- O `build.sh` garante que `app/index.html` existe
- O nome é `Dockerfile`: D maiúsculo e sem extensão

```
cd ~/meu-projeto-devops
./scripts/build.sh        # garante que app/index.html existe
vi Dockerfile
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 4 — Conteúdo do Dockerfile

- `FROM nginx:alpine` — parte da imagem oficial do nginx
- `COPY` — leva o seu `app/index.html` para dentro da imagem
- `EXPOSE 80` — documenta a porta

**A mesma receita da Parte 1, agora com a SUA página.**

```
FROM nginx:alpine
COPY app/index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 5 — Construa a imagem

- `-t meu-projeto:1.0` dá nome e tag (versão) à imagem
- O `.` final diz "a receita está nesta pasta"
- `docker images` mostra o bolo pronto no congelador local

```
docker build -t meu-projeto:1.0 .
docker images
```

Resultado esperado: a imagem `meu-projeto` com tag `1.0` na lista.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Rode a SUA aplicação em container 🚀

- Momento **uau**: abra **http://localhost:8080** no navegador do **seu computador**
- É a regra `WEB` de redirecionamento de porta do setup em ação
- Script → container → nginx → sua tela. Print no chat! 🎉

```
docker run -d --name meu-projeto -p 8080:80 meu-projeto:1.0
curl http://localhost:8080
```

Resultado esperado: a sua página — `Bootcamp DevOps Matera 2026` com a data do build.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 7 — Versione a receita

- O Dockerfile é código — e código vive no Git (Aula 06!)

**A receita agora faz parte da linha do tempo do projeto.**

```
git add Dockerfile
git commit -m "adiciona Dockerfile da aplicacao"
```

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 8 — Arrume a casa 🧹

- A porta 8080 precisa estar livre na próxima aula
- A imagem `meu-projeto:1.0` continua guardada (veja com `docker images`)
- É o bolo no congelador, pronto para novas fatias

```
docker stop meu-projeto
docker rm meu-projeto
docker ps -a
```

Resultado esperado: lista vazia no `docker ps -a`.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢 (1/2)

- `permission denied ... docker.sock` → faltou o `usermod` (Passo 1) ou sair/entrar de novo
- `DEPRECATED: The legacy builder is deprecated` → só um aviso; some com o `docker-buildx`
- `port is already allocated` → já existe container na 8080: `docker ps` e pare o antigo

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Deu errado? 😢 (2/2)

- `COPY failed: ... not found` → build fora da pasta do projeto ou sem `app/index.html`
- `Unable to find image` → é o download da imagem; na primeira vez demora, é normal

**A tabela completa está no pratica.md. Travou? Cola a mensagem no chat.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus 🏆

- Rode um **segundo** container da mesma imagem na porta 8081
- Use `--name meu-projeto-2 -p 8081:80`
- Confirme com `docker ps` e `curl`: duas fatias do mesmo bolo, isoladas, em segundos
- Depois limpe tudo (`docker stop` + `docker rm`)

**Uma imagem, vários containers: é o bolo rendendo várias fatias.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌 (1/2)

- 1. Suba o container do projeto, abra no navegador e derrube (`docker stop` + `docker rm`)
- Repita até fazer sem olhar

```
docker run -d --name meu-projeto -p 8080:80 meu-projeto:1.0
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Missão até a próxima aula 📌 (2/2)

- 2. Linux dentro do Linux: terminal **dentro** de um container Alpine (distribuição mínima)
- Lá dentro rode `ls /` e `cat /etc/os-release`; saia com `exit`

**Tudo que você aprendeu nas aulas de Linux vale ali dentro também.**

```
docker run --rm -it alpine sh
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

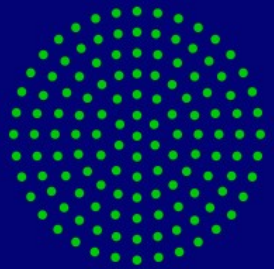
# O que levamos para a próxima aula

- Container = app + dependências numa caixa padrão que roda igual em qualquer lugar
- Dockerfile (receita) → imagem (bolo) → container (fatia)
- Você instalou o Docker, construiu uma imagem e serviu a sua página com nginx

**Um container, fácil. Sistemas reais têm vários: app, banco, cache... Na próxima aula, um arquivo orquestra tudo: docker compose.**

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 08

## Docker 2: Docker Compose

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda do encontro

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: Orquestrando containers
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa — um sistema inteiro com `docker compose`
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Objetivo de hoje: orquestrar vários containers com um único arquivo declarativo, o `compose.yaml`.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- Container = a carga na caixa padronizada: roda igual em qualquer lugar.
- Receita (**Dockerfile**) → bolo (**imagem**) → fatia servida (**container**).
- Vocês containerizaram a página do projeto e a serviram com `docker run -p 8080:80`.

**Gancho: foi só um container. Sistemas de verdade têm app + banco + cache... Hoje um único arquivo sobe tudo por nós.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## Orquestrando containers

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# A dor: o comando que não acaba mais

Imagine subir um sistema com 3 serviços na mão:

**Três comandos gigantes, ordem certa, parâmetros decorados. E amanhã, tudo de novo. 😫**

```
docker run -d --name banco -e SENHA=... -v dados:/var/lib/... rede...
docker run -d --name api -p 3000:3000 --link banco ...
docker run -d --name site -p 8080:80 ...
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Três comandos, todo dia

- **Ordem certa**: o banco antes da API, a API antes do site.
- **Parâmetros decorados**: senha, volume, rede, portas — um erro e nada sobe.
- **Amanhã, tudo de novo**: a mesma sequência, a cada vez que o sistema precisa subir.

💬 **Interação: quem já perdeu uma tarde subindo um ambiente 'na mão'? Conta no chat.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Declarar em vez de digitar

O **Docker Compose** lê um arquivo `compose.yaml` que **declara** o sistema inteiro — serviços, portas, volumes e dependências.

E o materializa com um único comando:

**Um arquivo descreve o sistema; um comando faz ele existir.**

```
docker compose up -d
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Declarativo vs. imperativo

- **Imperativo**: você dita os passos, um por um (os três `docker run`).
- **Declarativo**: você descreve o **estado desejado** e a ferramenta faz acontecer.
- Guarde essa ideia: é a base da Infraestrutura como Código e do Kubernetes.

💡 **Conceito-chave do DevOps: descrever o que se quer, não como fazer.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Anatomia de um compose.yaml (1/2)

- `services`: a lista de containers do sistema — cada serviço = um container.
- `build: .` usa o Dockerfile da pasta — o da Aula 07.

```
services:            # a lista de containers do sistema
  site:              # nome do serviço (você escolhe)
    build: .         # construa a partir do Dockerfile desta pasta
    ports:
      - "8080:80"    # VM:container — igual ao -p!
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Anatomia de um compose.yaml (2/2)

- `image:` usa uma imagem pronta do Hub, no lugar de `build:`.
- `redis:alpine` é o **Redis**, um banco de dados em memória — hoje, serviço de apoio.

```
visitas:
  image: redis:alpine   # imagem pronta do Hub
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# YAML: indentação é tudo

- **YAML**: formato de texto sensível a **indentação** — os espaços no início de cada linha.
- A indentação diz "isto está dentro daquilo": `ports` está dentro de `site`.
- 2 espaços por nível, **nunca Tab!**

**O erro mais comum da noite será de indentação. Olhe os espaços antes de culpar o Docker.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Serviços se enxergam pelo nome

- Cada serviço = um container.
- O Compose cria automaticamente uma **rede interna** entre os serviços.
- O `site` acessa o `visitas` pelo hostname `visitas` — sem decorar IP.

**DNS da Aula 03 em ação: nome vira endereço, automaticamente.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Comandos do dia a dia

- `docker compose up -d` — sobe o sistema inteiro (constrói o que precisar)
- `docker compose ps` — situação dos serviços
- `docker compose logs -f site` — acompanha os logs de um serviço
- `docker compose down` — derruba tudo, limpo

💬 **Interação: qual desses lembra um comando `docker` da Aula 07? Responde no chat.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Onde isso te leva: o time

- Ambiente de desenvolvimento de um time inteiro: `git clone` + `docker compose up`.
- Fim do "configura aí que na sexta eu te ajudo".

💬 **Interação: quanto tempo leva hoje para alguém novo rodar um sistema pela primeira vez? Chuta no chat.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Infraestrutura versionada

- O `compose.yaml` vive no Git — **infraestrutura versionada** junto com o código.
- Em produção, essa ideia cresce e vira **Kubernetes** (spoiler da Aula 10!).

**Declarativo hoje no Compose, declarativo amanhã no Kubernetes: a mesma ideia, em escala.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Hoje na prática

O projeto ganha um `compose.yaml` com **dois serviços** — sua página e um serviço de apoio (Redis) — subindo juntos com um comando.

- Instalar o plugin do Compose
- Declarar o sistema e subir tudo
- Espiar logs, provar a rede interna, derrubar e recriar
- Versionar a infraestrutura no Git

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Mão na massa

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Objetivo e onde

**Objetivo:** declarar um sistema com dois serviços — a página do seu projeto + um banco Redis de apoio — e subir tudo com um único comando.

**Onde:** no terminal conectado à VM via SSH (como no setup). A janela do VirtualBox também serve, mas sem copiar e colar.

**Confirme no chat: VM ligada e sessão SSH aberta?**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — Instale o plugin do Compose

- O Compose é um **plugin** do Docker: um pacote a mais, o `docker-compose-v2`.
- `docker compose version` confirma que o plugin foi reconhecido.

```
sudo apt update
sudo apt install -y docker-compose-v2
docker compose version
```

Resultado esperado: `Docker Compose version 2.x...`.

REALIZAÇÃO:
Instituto MATERA

# Passo 2 — Declare o sistema

- Entre na pasta do projeto: é onde está o `Dockerfile` da Aula 07.
- Crie o arquivo `compose.yaml` com o `vi` (próximo slide: o conteúdo).
- No `vi`: `i` para inserir, `Esc` e `:wq` para salvar e sair.

```
cd ~/meu-projeto-devops
vi compose.yaml
```

Atenção à indentação — 2 espaços, nunca Tab! Salve com `:wq`.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 2 — O compose.yaml

- `site`: do seu `Dockerfile` (Aula 07), porta 8080; `visitas`: imagem oficial do Redis.
- `depends_on`: o `site` só sobe depois do `visitas`.

```
services:
  site:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - visitas
  visitas:
    image: redis:alpine
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Suba o sistema inteiro 🚀

- Na primeira vez o Compose constrói a imagem do `site` e baixa o `redis:alpine`.
- `docker compose ps` lista os dois serviços e o estado de cada um.
- No navegador do seu computador: **http://localhost:8080**.

```
docker compose up -d
docker compose ps
curl http://localhost:8080
```

Resultado esperado: dois serviços com status `Up`/`running` e a sua página no `curl` e no navegador.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 4 — Espie os bastidores

- Logs de cada serviço, chamando pelo nome do serviço.
- Quer ver o log inteiro? `docker compose logs visitas`.
- O aviso `WARNING Memory overcommit must be enabled!` é inofensivo aqui.

```
docker compose logs visitas | grep Ready
```

Resultado esperado: a linha `Ready to accept connections tcp` do log do Redis.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — A mágica da rede interna

- `exec site` entra no container do site e roda um comando lá dentro.
- O `ping` chama o `visitas` pelo **hostname** — nenhum IP decorado.

**Os serviços se enxergam pelo nome.**

```
docker compose exec site ping -c 2 visitas
```

Resultado esperado: o `visitas` responde! Rede privada com DNS automático (Aula 03 mandou lembranças 📡).

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — Dados vivendo no serviço

- `redis-cli` é o cliente do Redis; `INCR` incrementa um contador, `GET` lê o valor.
- Rode o `INCR` mais vezes e veja o contador subir.
- Dados vivendo em um serviço, consultados de fora dele.

```
docker compose exec visitas redis-cli INCR visitas_do_site
docker compose exec visitas redis-cli GET visitas_do_site
```

Resultado esperado: `(integer) 1` e, no `GET`, `"1"`.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 6 — Derrube e suba de novo (1/2)

- Um comando derruba o sistema; outro o recria.

**Esse é o superpoder do declarativo: o arquivo é a verdade, o ambiente é descartável.**

```
docker compose down
docker compose ps
docker compose up -d
```

Resultado esperado: `ps` vazio depois do `down`; depois do `up -d`, tudo de volta em segundos.

REALIZAÇÃO:
Instituto MATERA

# Passo 6 — Derrube e suba de novo (2/2)

- Os dados viviam dentro do container: ambiente descartável, dados descartados.
- Para dados sobreviverem, usa-se um **volume** — tema para explorar depois do bootcamp.

```
docker compose exec visitas redis-cli GET visitas_do_site
```

Resultado esperado: `(nil)` — o contador sumiu. Não é bug: o Redis foi destruído e recriado do zero.

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 7 — Versione a infraestrutura

- O `compose.yaml` é texto: entra no Git como qualquer arquivo do projeto.
- Seu repositório conta a história completa: app, scripts, receita da imagem e sistema. 📜

```
git add compose.yaml
git commit -m "adiciona compose.yaml com site e redis"
git log --oneline
```

Resultado esperado: o novo commit no topo do `git log --oneline`.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Deu errado? 😢

- `yaml: line X: ...` → indentação errada: use 2 espaços, remova Tabs
- `port is already allocated` → container da Aula 07 na 8080: `docker ps` + `docker stop`
- `permission denied ... docker.sock` → sessão sem o grupo docker: `exit` e volte pelo SSH
- `no configuration file provided` → rode de dentro de `~/meu-projeto-devops`
- `'compose' is not a docker command` → plugin não instalado: refaça o Passo 1
- `pull access denied` → erro de digitação no nome da imagem (`redis:alpine`)

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Desafio bônus 🏆

- Adicione um terceiro serviço ao `compose.yaml`: imagem `traefik/whoami`, porta `8081:80`.
- Suba com `docker compose up -d` — só o novo serviço será criado!
- Teste com `curl http://localhost:8081`.

**Declarou mais um serviço, rodou o mesmo comando: o Compose só cria o que falta.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Missão até a próxima aula 📌

Na Aula 09 o seu projeto vai para o GitHub — e criar a conta no meio da aula consome tempo precioso. Faça antes:

- 1. Crie sua conta em **https://github.com/signup** — nome de usuário profissional.
- 2. Confirme o e-mail.
- 3. Recomendado: verificação em duas etapas (**Settings → Password and authentication**).

**Já tem conta? Missão cumprida.** 😉

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

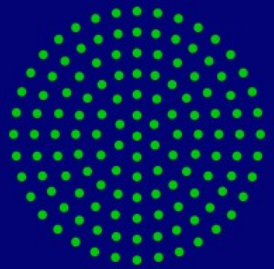
# O que levamos para a próxima aula

- Um `compose.yaml` declara o sistema; `docker compose up -d` materializa tudo de uma vez.
- Declarativo vs. imperativo: descrever o estado desejado, não ditar os passos.
- Rede interna com DNS: serviços se enxergam pelo nome; infra versionada no Git.

**Tudo commitado, mas preso na sua VM. Se ela morrer, morre junto. E o recrutador, como vê? Próxima aula: GitHub.**

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 09

## GitHub: seu código na nuvem, seu portfólio no mundo

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda de hoje

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: Git não é GitHub (e por que você precisa dos dois)
- 20:20 — Intervalo (15 min)
- 20:35 — Parte 2: Mão na massa — publicando seu projeto no GitHub
- 21:20 — Perguntas & Respostas
- 21:45 — Encerramento

**Hoje: primeiro** `push`**, primeiro Pull Request e o começo do seu portfólio.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- `compose.yaml` declara o sistema inteiro; `docker compose up -d` materializa
- Declarativo > imperativo
- Serviços se enxergam pelo nome na rede interna
- A infraestrutura está **versionada no Git**

**Gancho: todo esse trabalho está preso na VM. Hoje ele vai para a nuvem — e vira vitrine: GitHub.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## Git não é GitHub: por que os dois?

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Git não é GitHub

- **Git** = a ferramenta de controle de versão — roda na sua máquina (Aula 06)
- **GitHub** = a **plataforma na nuvem** que hospeda repositórios Git
- E adiciona colaboração: Pull Requests, revisão de código, issues, automação (Actions!)
- ... e visibilidade: o mundo pode ver o seu trabalho

**Git você já usa desde a Aula 06. Hoje entra o GitHub: os dois juntos.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Editor de texto × biblioteca

**Git** é o editor de texto: onde você escreve e guarda cada versão da obra.

**GitHub** é a biblioteca pública: onde a obra fica disponível, com histórico de edições e espaço para revisores.

**Sem o GitHub, seu projeto é um livro guardado na gaveta — só você lê.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Remotos: o elo local e nuvem

- **Remoto** = a cópia do repositório que vive no GitHub (o nome padrão é `origin`)
- `git push` envia seus commits para lá; `git pull` traz os commits novos de lá

**Local e remoto são dois repositórios Git completos — o push e o pull mantêm os dois em sincronia.**

```
[ Repo local (VM) ]  --git push-->  [ Repo remoto (GitHub) ]
                     <--git pull--
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Os comandos de remoto

- `git remote add origin <url>` — aponta o repositório local para o remoto
- `git push` — envia seus commits para o GitHub
- `git pull` — traz os commits novos do GitHub
- `git clone <url>` — baixa um repositório inteiro

**Hoje você vai usar os três primeiros na prática — e o `clone` fica na manga.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# git clone: qualquer repo público

`git clone <url>` baixa um repositório inteiro — código e todo o histórico de commits.

Funciona com **qualquer repositório público** do GitHub, não só com os seus.

**É assim que você pega o código de um projeto que não começou na sua máquina.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Pull Request: o pedido formal

O **PR** (Pull Request) é o pedido formal: "revisem minha branch e, se estiver boa, integrem na main".

É a cerimônia mais importante do dia a dia de quem trabalha com código.

- Você trabalha numa **branch** (Aula 06), não direto na `main`
- Quando termina, abre o PR: o time olha antes de integrar

**PR não é burocracia: é o momento em que o trabalho vira trabalho de time.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que acontece no PR

- **Revisão de código** — outro par de olhos antes de integrar
- **Discussão registrada** — o porquê das decisões fica documentado
- **Verificações automáticas** — a pipeline roda **no PR**, antes do merge

**Amanhã você fará exatamente isso: a pipeline da Aula 05 rodando dentro do PR.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PR e cultura blameless

No PR, a revisão aponta o que pode melhorar no **código** — nunca julga a **pessoa**.

É a cultura blameless (sem culpados) da Aula 01 em ação, todos os dias.

💬 **Interação: Lembra da cultura blameless da Aula 01? O PR é ela em ação: crítica ao código, nunca à pessoa.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Autenticação segura

- O GitHub **não aceita senha** em `git push`
- Usaremos um **Personal Access Token (PAT)**: um token de acesso pessoal
- É uma senha gerada pelo GitHub, com prazo e permissões limitadas
- Alternativa: chave **SSH** (não usaremos hoje)
- Hoje: PAT **fine-grained** (escopo restrito), só para o `meu-projeto-devops`

**Token = senha com prazo de validade e alcance limitado. Hoje: 30 dias, um único repositório.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Regra de ouro: token é segredo

- **Nunca** commite um token
- **Nunca** cole um token em arquivo do projeto
- **Nunca** compartilhe print com token visível

🔐 **DevSecOps: trate o token como escova de dentes — só sua e trocada com frequência.** 🪥

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# GitHub como portfólio

Recrutadores de tecnologia **olham o GitHub**.

Um perfil com projetos reais, commits com mensagens caprichadas e README bem escrito vale mais que dez cursos listados.

**Seu GitHub é a vitrine que fala por você antes da entrevista.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Seu projeto vira vitrine

- O `meu-projeto-devops` que você constrói desde a Aula 02 vira hoje sua peça de portfólio
- Scripts, Dockerfile, compose, histórico de commits — tudo vai para a nuvem
- Na Parte 2: conta, repositório, token, primeiro `push` e primeiro Pull Request

**Hoje à noite, o seu portfólio começa.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Publicando no GitHub ☁️

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Objetivo e onde

**Objetivo:** criar sua conta e repositório no GitHub, enviar o `meu-projeto-devops` para a nuvem e abrir seu primeiro Pull Request.

- **Onde:** navegador do **seu computador** + terminal conectado à VM **via SSH**
- SSH é obrigatório hoje: você vai colar um token longo (setup, passo 7)
- Troque `SEU-USUARIO` pelo seu usuário do GitHub em todos os comandos

**Ciclo de hoje: conta → repositório → token → push → branch → PR → merge → pull.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — Conta no GitHub pronta?

- Criou a conta na missão da Aula 08? Siga direto para o Passo 2. 😉
- Ainda não? Crie agora em **https://github.com/signup**
- Nome de usuário **profissional** — ele aparecerá no seu portfólio — e confirme o e-mail
- Se o cadastro pedir alguma verificação, chame um monitor para não perder o ritmo

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 2 — Crie o repositório

No GitHub: botão **New** (ou https://github.com/new) e preencha:

- Repository name: `meu-projeto-devops`
- Description: Projeto do Bootcamp DevOps Matera 2026
- Choose visibility: **Public** (é portfólio!)
- Add README: **Off** · Add .gitignore: **No .gitignore** · Add license: **No license**

**Clique no botão "Create repository".**

Resultado esperado: o GitHub abre a página do repositório `meu-projeto-devops`, ainda vazio.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Crie um token (PAT) (1/2)

- Avatar → **Settings** → **Developer settings** → **Personal access tokens**
- → **Fine-grained tokens** → **Generate new token**
- Token Name: `bootcamp-matera` · Description: `Token bootcamp-matera`
- Resource owner: seu username · Expiration: `30 dias`

**Próximo slide: o alcance do token (repositório e permissões).**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Crie um token (PAT) (2/2)

- Repository access: **Only select repositories** → `meu-projeto-devops`
- **+ Add permissions**: **Contents: Read and write** e **Workflows: Read and write**

**Workflows é obrigatória na Aula 10, quando você enviará a pipeline.**
**Copie o token: ele aparece uma única vez!**

🔐 O token é sua senha para esta VM. Não o salve em arquivos do projeto, não poste print dele no chat.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 4 — Conecte local ao remoto

- Troque `SEU-USUARIO` pelo seu usuário do GitHub
- `git remote add origin` aponta o repositório local para o remoto na nuvem
- `git remote -v` confere: deve listar `origin` com a URL (fetch e push)

```
cd ~/meu-projeto-devops
git remote add origin https://github.com/SEU-USUARIO/meu-projeto-devops.git
git remote -v
```

Resultado esperado: `origin` listado duas vezes (fetch e push), com a URL do seu repositório.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — O primeiro push! 🚀

- Quando pedir usuário: seu usuário do GitHub
- Quando pedir senha: **cole o token** (nada aparece na tela — é normal; dê Enter)
- Atualize a página do repositório no navegador: **seu projeto está na nuvem!** 🎉

```
git push -u origin main
```

Resultado esperado: `main -> main` e, por último, `branch 'main' set up to track 'origin/main'`.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Para não colar o token a cada push

- ⚠️ Grava o token **em texto puro** em `~/.git-credentials` da VM
- Aceitável aqui: VM de estudo, token de 30 dias para um único repositório
- No trabalho, prefira `git config --global credential.helper 'cache --timeout=7200'`

```
git config credential.helper store
```

O cache guarda o token só na memória, por 2 horas. Ao fim do bootcamp, revogue o token em Developer settings.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Branch e README.md

O GitHub mostra automaticamente o arquivo `README.md` na página do repositório. Vamos renomear o `README.txt` da Aula 02 e reescrevê-lo.

- `git checkout -b` cria a branch `docs-readme`: a melhoria não vai direto na `main`
- `git mv` renomeia e já avisa o Git

```
git checkout -b docs-readme
git mv README.txt README.md
vi README.md
```

Resultado esperado: você está na branch `docs-readme`, com o `README.md` aberto no vi.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Escreva em markdown (1/2)

Apague o que estava lá e escreva um README caprichado em **markdown**: `#` vira título e `-` vira lista. Por exemplo:

```
# Meu Projeto DevOps
Projeto construído no Bootcamp DevOps Matera 2026.
## O que tem aqui
- `scripts/` — health check e pipeline (build, test, deploy)
- `Dockerfile` — imagem da aplicação
- `compose.yaml` — app + Redis com docker compose
```

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Escreva em markdown (2/2)

- Feche o README com a seção de como rodar o projeto:

```
## Como rodar
docker compose up -d
```

Salve e saia do vi (`:wq`). O exemplo completo, com linhas em branco entre as seções, está no pratica.md.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Commit e push da branch

- A branch `docs-readme` sobe para o GitHub — a `main` continua intacta
- Se pedir senha de novo, cole o token (ou já está salvo pelo credential helper)

```
git add README.md
git commit -m "adiciona README do projeto"
git push -u origin docs-readme
```

Resultado esperado: push concluído; a branch `docs-readme` já aparece no GitHub.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 7 — Abra o PR

- Logo após o push, o GitHub mostra um aviso amarelo **Compare & pull request** — clique
- Sumiu? Aba **Pull requests** → **New pull request** → em **compare**, escolha `docs-readme`
- Título: `Adiciona README do projeto`. Descreva **o que** e **por quê**
- **Create pull request**

**Explore a aba Files changed — é assim que um colega revisaria seu código.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 7 — Integre e sincronize

- **Merge pull request → Confirm merge**
- De volta à VM, sincronize a `main` local:

**Ciclo completo: branch → push → PR → review → merge → pull. Esse é o fluxo de milhões de devs, e agora é o seu.**

```
git checkout main
git pull
```

Resultado esperado: o README integrado chegou à sua `main` local.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Deu errado? 😢 (1/2)

- `Authentication failed` → token errado/expirado ou senha da conta; gere/cole o PAT
- `remote origin already exists` → `git remote remove origin` e refaça o Passo 4
- `src refspec main does not match any` → branch é `master`; rode `git branch -M main`

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢 (2/2)

- Colei o token e nada apareceu → normal, senhas não aparecem; dê Enter
- `failed to push some refs` → remoto tinha README; pull com `--allow-unrelated-histories`
- `Repository not found` → erro de digitação na URL (usuário/repo); confira `git remote -v`

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus 🏆

- Foto de perfil
- Bio: "Estudante de DevOps — Bootcamp Matera 2026"
- Fixe (pin) o repositório `meu-projeto-devops` na página inicial do perfil

**Capriche no seu perfil do GitHub: é a sua vitrine a partir de agora.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão até a próxima aula 📌

- 1. Faça o desafio bônus (perfil + repositório fixado): é a sua vitrine a partir de agora
- 2. Abra um **segundo PR** sozinho(a): corrija ou melhore qualquer coisa no `README.md`
- Pelo fluxo completo: branch → commit → push → PR → merge → `git pull`

**Na próxima aula, a pipeline vai rodar dentro desse mesmo fluxo.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PERGUNTAS & RESPOSTAS

21:20 – 21:45

REALIZAÇÃO:

Instituto MATERA

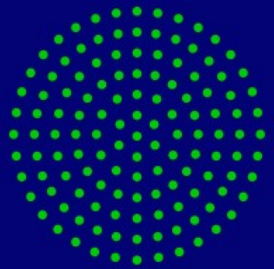
UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que levamos para a próxima aula

- Git é a ferramenta local; GitHub é a plataforma na nuvem (remotos: `push` e `pull`)
- Pull Request: revisão, discussão registrada e verificações automáticas antes do merge
- Token é segredo (nunca no projeto, nunca em print) — e seu projeto já é portfólio

**Gancho: seu projeto está no GitHub, com PR e tudo. Falta o robô que builda e testa a cada push: GitHub Actions.** 🎁



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula 10

## GitHub Actions + Encerramento 🏁

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Agenda de hoje

- 19:30 — Abertura e boas-vindas
- 19:35 — Parte 1: GitHub Actions, a esteira dentro do repositório
- 20:20 — Intervalo
- 20:35 — Parte 2: Mão na massa (a última!)
- 21:20 — Aula extra: a surpresa final 🎁 (no lugar do Q&A)
- 21:45 — Encerramento

**Objetivo: fechar o ciclo — a cada push, o GitHub builda e testa o seu projeto sozinho.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Revisão da aula anterior

- Git local + GitHub remoto: `push`, `pull` e o fluxo branch → PR → review → merge
- O `meu-projeto-devops` está público, com README — portfólio no ar

**Gancho: lembra da pipeline manual da Aula 05, em que você era o robô? Hoje o robô assume.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 1

## A esteira dentro do repo

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# O que é o GitHub Actions

O serviço de **CI/CD embutido no GitHub**: você descreve a pipeline em um arquivo YAML dentro do próprio repositório.

O GitHub executa em máquinas dele — os **runners** —, de graça para repositórios públicos (como o seu `meu-projeto-devops`).

- YAML: texto de configuração; runner: a máquina que roda a pipeline
- O arquivo mora em `.github/workflows/ci.yml`, versionado com `git`
- Mudou a pipeline? Vira commit, com histórico — quem clona leva junto

💡 **Pipeline versionada junto com o código = a fonte única da verdade da Aula 06 valendo também para a automação.**

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Anatomia: o gatilho

- `name`: o nome que aparece na aba Actions
- `on`: QUANDO rodar — a cada `push` na `main` e em todo `pull_request`
- Na Aula 05 o gatilho era você digitando `./pipeline.sh`

```
name: CI                          # nome da pipeline

on:                               # QUANDO rodar (o gatilho)
  push:
    branches: [main]
  pull_request:
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Anatomia: jobs e steps

- `runs-on: ubuntu-latest`: o runner é um Ubuntu — os mesmos comandos da sua VM
- `uses`: uma action pronta (passo já feito por alguém); `run`: um comando seu

```
jobs:                          # O QUE rodar
  build-e-teste:
    runs-on: ubuntu-latest     # em qual máquina (olha o Linux aí! 🐧)
    steps:                     # os passos, em ordem
      - uses: actions/checkout@v4      # 1) baixa o código do repo
      - name: Build
        run: ./scripts/build.sh        # 2) roda SEU script de build
      - name: Teste
        run: ./scripts/test.sh         # 3) roda SEU script de teste
```

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Traduzindo o vocabulário

- **workflow** — a pipeline inteira — equivale ao `pipeline.sh` da Aula 05
- **trigger** (`on`) — o evento que dispara — equivale a você digitando `./pipeline.sh`
- **job** — grupo de passos numa máquina
- **step** — um passo (comando ou action pronta) — equivale a `build.sh`, `test.sh`
- **runner** — a máquina que executa — equivale à sua VM

💬 **Interação: no chat, qual peça da Aula 05 virou o 'runner' hoje?**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# O ciclo completo

- Verde: o repositório ganha um selo (o badge) e o PR pode ser integrado
- Vermelho: você é avisado na hora e o erro não chega à `main`

**Isso é o que você construiu em 10 aulas: um ciclo de integração contínua de ponta a ponta.**

```
git push → GitHub Actions acorda → checkout → build → testes
   → ✅ verde: badge verde no repo, PR liberado para merge
   → ❌ vermelho: e-mail na hora, PR sinalizado como inseguro,
        produção protegida
```

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Avisar não é bloquear

Por padrão, o GitHub **avisa** que os checks (as verificações) falharam, mas ainda deixa o botão de merge ativo.

- Para **bloquear** de verdade: Settings → Rules → Rulesets → New branch ruleset
- Marque **Require status checks to pass**
- Em times reais isso é regra, não opção

🔒 **Check vermelho + botão de merge ativo: hoje é você quem decide não clicar. No time, a regra decide.**

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Hoje tudo se conecta

- Linux (Aulas 02–04): o runner é um Ubuntu e roda os seus comandos
- Pipeline (Aula 05): `build.sh` e `test.sh` são os steps
- Git (Aula 06): o workflow é versionado com o código
- Containers (Aulas 07–08): a imagem e o compose seguem no projeto
- GitHub (Aula 09): o push dispara tudo e o PR mostra o resultado

**Cada peça dessa frase você aprendeu numa aula. Hoje tudo se conecta.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Segredos na pipeline

E quando a pipeline precisa de uma senha — um registro de imagens (registry), a nuvem? **Nunca no YAML!**

- O GitHub tem um cofre: Settings → Secrets and variables → Actions
- No workflow, usa-se `${{ secrets.MINHA_SENHA }}`
- O valor nunca aparece nos logs

**DevSecOps na prática: o segredo fica no cofre do GitHub e o workflow só o chama pelo nome.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Próximos passos: K8s e DevSecOps

- **Kubernetes (K8s):** o compose 'adulto' — orquestra containers em **muitas máquinas**
- Auto-recuperação e escala automática; padrão da indústria e dia a dia na Matera
- **DevSecOps:** segurança desde o dia zero
- Scanners de vulnerabilidade e de segredos **como stages (etapas) da pipeline**

**Tudo o que vem depois é a mesma lógica de hoje, em escala maior.**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Próximos passos: a jornada continua

- **Infraestrutura como Código (Terraform)**, observabilidade (métricas, logs, traces)
- Nuvem (AWS/Azure/GCP) — cada um desses dá um bootcamp inteiro
- **Continue no Linux todos os dias:** o guia continuando com Linux
- Na VM, com WSL2 no Windows ou no Terminal do Mac — com um desafio de 30 dias
- A cola de comandos (cheat-sheet-terminal) para ter sempre à mão

**Terminal é como idioma: quem não usa, esquece. O guia traz um desafio de 30 dias.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Carreira e comunidade

- Seu diferencial: um repositório público com **projeto funcional e pipeline verde**
- Isso é mais do que muito candidato júnior tem
- Compartilhe o link do seu repositório com quem quiser — ele é o seu portfólio
- Continuem nos canais da comunidade; as portas do Instituto Matera seguem abertas

💬 **Interação: o que você vai automatizar primeiro depois do bootcamp? Conta no chat.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# INTERVALO

Voltamos às 20:35

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# PARTE 2

## Mão na massa (a última!)

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Objetivo e onde

**Objetivo:** fazer o GitHub buildar e testar seu projeto **automaticamente a cada push**.

A pipeline da Aula 05, agora executada por um robô de verdade.

- **Onde:** terminal conectado à VM **via SSH** (necessário para colar o badge)
- \+ navegador com o GitHub aberto e logado
- Roteiro completo em `aulas/aula-10-github-actions/pratica.md`

**Sete passos: criar o workflow, ver a primeira pipeline verde, quebrar e consertar — blameless (sem culpados) até o fim!**

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 1 — Crie o arquivo do workflow

- O caminho é sagrado: `.github/workflows/` (com ponto, no plural!)
- `mkdir -p` cria as duas pastas de uma vez
- O `vi` abre o arquivo vazio: `i` para inserir, `Esc` e `:wq` para salvar

```
cd ~/meu-projeto-devops
mkdir -p .github/workflows
vi .github/workflows/ci.yml
```

Resultado esperado: o vi aberto em um arquivo novo dentro de .github/workflows/.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — ci.yml (1/2)

- Indentação com **2 espaços** — sem Tab!
- Digite com calma: o nome do job é `build-e-teste`, com hífens

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build-e-teste:
```

Resultado esperado: primeira metade digitada; a segunda vem no próximo slide.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 1 — ci.yml (2/2)

- 💡 Reutiliza **os scripts que você escreveu na Aula 05**
- Salve com `Esc` e `:wq`

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build
        run: ./scripts/build.sh

      - name: Teste
        run: ./scripts/test.sh
```

Resultado esperado: arquivo salvo, igual ao modelo do pratica.md.

REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 2 — Commit e push

- O mesmo ritual da Aula 06: `add`, `commit`, `push`
- O `push` vai pedir o usuário e o token do GitHub, como na Aula 09

```
git add .github/workflows/ci.yml
git commit -m "adiciona pipeline de CI com GitHub Actions"
git push
```

Resultado esperado: o push termina sem erro e o commit aparece no repositório no navegador.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 3 — Veja o robô trabalhar 🍿

- No navegador: seu repositório → aba **Actions**
- O workflow `CI` aparece rodando (bolinha amarela)
- Clique nele → clique no job `build-e-teste` → expanda os steps **Build** e **Teste**

**Os mesmos `[BUILD] OK` e `[TEST] OK` da VM, agora numa máquina do GitHub, disparados pelo seu push. Essa é a Integração Contínua.**

Resultado esperado: ✅ verde, com [BUILD] OK e [TEST] OK nos logs dos steps.

REALIZAÇÃO:

Instituto MATERA

# Passo 4 — O selo no README (opcional)

- **Se der tempo; senão, vira missão de casa**
- Na página do workflow: botão `...` → **Create status badge** → copie o markdown
- Por isso o SSH: colar o badge do navegador no `vi` da VM

```
vi README.md    # cole o badge na primeira linha
git add README.md
git commit -m "adiciona badge de status da CI"
git push
```

Resultado esperado: atualizando a página do repositório, um selo passing no topo. Portfólio com prova de qualidade. 😎

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 5 — Quebre de propósito 💥

- O gran finale: integrar um código quebrado pelo fluxo da Aula 09 e ver a esteira barrar
- No `vi`, troque `Bootcamp DevOps Matera 2026` por `Ola mundo`
- O teste vai reprovar — lembra? Ele confere o conteúdo da página

```
git checkout -b quebra-propositaL
vi scripts/build.sh
```

Resultado esperado: branch quebra-propositaL criada e build.sh alterado com o título errado.

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Passo 5 — A esteira barra

- No navegador, abra o **Pull Request** dessa branch
- Em segundos, o check **CI / build-e-teste** fica ❌ vermelho, **dentro do PR**
- Clique em **Details** e encontre a linha exata do `[TEST] FALHOU`

**O GitHub avisa que o merge não é seguro antes de o erro chegar à main. O botão segue ativo: você decide não clicar.**

```
git add scripts/build.sh
git commit -m "altera titulo da pagina"
git push -u origin quebra-propositalgit
```

Resultado esperado: PR aberto com o check vermelho e o [TEST] FALHOU em Details.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 6 — Conserte e veja o PR verde

- O PR reexecuta a pipeline sozinho → ✅ verde → **Merge pull request**
- Na VM: `git checkout main && git pull`

**Ciclo DevOps completo, de ponta a ponta, no seu projeto.** 🏆

```
vi scripts/build.sh     # desfaça a mudança
git add scripts/build.sh
git commit -m "corrige titulo da pagina"
git push
```

Resultado esperado: check verde no PR, merge feito e a main local atualizada com o pull.

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Passo 7 — Guarde o seu portfólio 🏅

- Anote o link do seu repositório
- Tire um print da pipeline verde na aba **Actions**
- Esse é o seu portfólio, construído por você em 10 aulas

**O certificado de participação será enviado pelo Instituto Matera, conforme as orientações da organização.**

Resultado esperado: link do repositório anotado e print da pipeline verde salvo.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Deu errado? 😢

- Aba Actions vazia → caminho exato `.github/workflows/ci.yml`; `ls -la .github/workflows/`
- `Permission denied` no Build → sem `+x`; `git update-index --chmod=+x scripts/*.sh`
- Erro de YAML → indentação: 2 espaços, sem Tab; compare com o modelo
- Badge não aparece → regenere pelo botão **Create status badge**
- `refusing to allow a Personal Access Token...` → token sem permissão **Workflows**

Token: Contents + Workflows: Read and write. Token antigo guardado? Apague a linha em ~/.git-credentials com o vi.

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Desafio bônus

- Transforme o aviso em bloqueio: Settings → Rules → Rulesets → New branch ruleset
- Marque **Require status checks to pass** para a `main`
- Reabra um PR quebrado e veja o botão de merge desativado

🔒 **Em times reais isso é regra, não opção.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Missão para depois do bootcamp 📌

- Leia o guia continuando com Linux e escolha o seu caminho
- Manter a VM, WSL2 no Windows ou o Terminal do Mac
- Aceite o **desafio dos 30 dias**: 10 minutos de terminal por dia
- Daqui a um mês, conte na comunidade o que você automatizou
- Badge no README (Passo 4) para quem não fez em aula

**O loop não termina — recomeça.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# AULA EXTRA

## A surpresa final · 21:20

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# O que levamos para a vida 🔗

Vocês chegaram sem saber o que era um terminal. Saem com Linux, scripts, Git, Docker, GitHub e uma esteira de CI num projeto público.

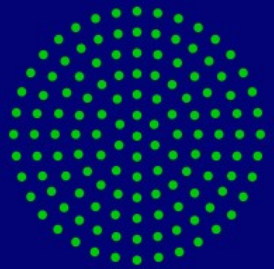
- Uma pipeline de verdade: a cada push, o GitHub builda e testa sozinho
- Segredos no cofre, nunca no YAML; o check vermelho protege a `main`
- Guia continuando com Linux: seu caminho e o desafio dos 30 dias

**O loop do DevOps não termina — ele recomeça: planejem o próximo aprendizado, codem, testem, monitorem a evolução de vocês.**

Certificado enviado pela organização. A comunidade fica. Obrigado, e até a próxima! 💙

REALIZAÇÃO:

Instituto MATERA



Instituto

MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Aula Extra

## IA e DevOps: saber o que pedir

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Dez aulas… ou um prompt?

Durante duas semanas, vocês construíram com as próprias mãos: projeto, health check, pipeline, Dockerfile, compose, GitHub e CI.

- Agora, ao vivo: vamos pedir isso a uma IA que roda no terminal
- Duas vezes — de dois jeitos diferentes

💬 **Enquete no chat: vocês levaram 10 aulas para construir esse projeto. Quanto tempo uma IA levaria?**

REALIZAÇÃO:
Instituto MATERA

# Prompt 1 — o pedido simples

- Linguagem natural, com o vocabulário do bootcamp — e pedindo a prova

```
Crie um projeto chamado meu-projeto-devops com scripts em bash:
- um build que gera uma pagina index.html com a data do build;
- um teste que verifica a pagina e falha com codigo de saida diferente de zero se
ela estiver errada;
- um deploy que copia a pagina para uma pasta de "producao";
- uma pipeline que roda build, teste e deploy nessa ordem e para no primeiro erro.
Inclua um Dockerfile com nginx para servir a pagina, um compose com o site e um
redis,
e um workflow do GitHub Actions que roda build e teste a cada push.
Rode a pipeline para provar que funciona e depois quebre o teste de proposito
para mostrar que o deploy nao acontece. Por fim, inicie o repositorio git e faca o
primeiro commit.
```

UPLOADS DE CONHECIMENTO
BOOT CAMP
REALIZAÇÃO:
Instituto MATERA

UPLOADS DE CONHECIMENTO

BOOT CAMP

# Funcionou... mas é o nosso projeto?

- Pasta `dist/` em vez de `app/`; produção dentro do projeto em vez de `~/producao`
- Título "Meu Projeto DevOps"; mensagens `[build]`/`[test]`; cinco verificações no teste
- `nginx:1.27-alpine`; serviço `redis` com volume e `restart`; workflow em qualquer push
- `.gitignore` e README que ninguém pediu

💬 **Interação: é o nosso projeto? O que está diferente? Digitem no chat.**

REALIZAÇÃO:

Instituto MATERA

UPLOADS DE CONHECIMENTO
BOOT CAMP

# Prompt 2 — a especificação

- 63 linhas: os 11 arquivos, nomes, caminhos, mensagens `[OK]`/`[FALHA]`, `set -e`, `grep -q`, `nginx:alpine`, serviço `visitas`, `ci.yml`
- Resultado (ensaiado): **igual ao projeto da turma** — até o `test.sh`
- Mesma IA, mesmo tempo. Mudou só uma coisa: **o pedido**

**A diferença entre os dois prompts é exatamente o que vocês aprenderam em dez aulas.**

Prompt completo em aulas/aula-10-github-actions/aula-extra-ia/prompt-detalhado.txt — mostre na tela e dê zoom em 2 ou 3 trechos.

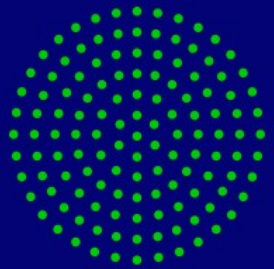
UPLOADS DE CONHECIMENTO
BOOT CAMP

# A virada

- **A IA só fez em minutos porque VOCÊS agora sabem julgar o que ela fez**
- Sem as 10 aulas, o resultado seria um bloco de texto indecifrável
- **O trabalho do DevOps mudou de lugar:** entender, revisar, decidir e operar
- A IA escreve o script; quem responde pelo deploy em produção é você
- **Quem domina os fundamentos multiplica a produtividade com IA**
- Quem não domina só copia e cola — e não sabe consertar quando quebra (e sempre quebra!)

💬 **Interação: a IA errou algo? O que vocês fariam diferente? Revisem a IA no chat.**

REALIZAÇÃO:
Instituto MATERA



Instituto

MATERA